In [1]:
```
import numpy as np
import pandas as pd
import seaborn as sns

from IPython.display import HTML
from IPython.display import display
import matplotlib.pyplot as plt

from sklearn.neighbors import KNeighborsClassifier
from sklearn.compose import ColumnTransformer, make_column_transformer
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.model_selection import cross_val_score, cross_validate
from sklearn.metrics import confusion_matrix

from sklearn.impute import SimpleImputer
from sklearn.metrics import ConfusionMatrixDisplay

from sklearn.metrics import mean_squared_error, mean_absolute_error

from sklearn.metrics import roc_curve, roc_auc_score, precision_recall_curve, a
from sklearn.calibration import calibration_curve

from sklearn.metrics import classification_report
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_s

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, learning_curve
from sklearn.metrics import accuracy_score

from sklearn.preprocessing import (
    MinMaxScaler,
    OneHotEncoder,
    OrdinalEncoder,
    StandardScaler
)

import warnings
warnings.filterwarnings('ignore')
```

In [2]:
```
df = pd.read_csv("healthcare_dataset.csv")
df.head()
```

Out[2]:

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	
0	Tiffany Ramirez	81	Female	O-	Diabetes	2022-11-17	Patrick Parker	Wallace-Hamilton	Medicare	3
1	Ruben Burns	35	Male	O+	Asthma	2023-06-01	Diane Jackson	Burke, Griffin and Cooper	UnitedHealthcare	4
2	Chad Byrd	61	Male	B-	Obesity	2019-01-09	Paul Baker	Walton LLC	Medicare	3
3	Antonio Frederick	49	Male	B-	Asthma	2020-05-02	Brian Chandler	Garcia Ltd	Medicare	2
4	Mrs. Brandy Flowers	51	Male	O-	Arthritis	2021-07-09	Dustin Griffin	Jones, Brown and Murray	UnitedHealthcare	1

In [3]:
```
df.shape
```

Out[3]:
```
(10000, 15)
```

In [4]:
```
df.describe()
```

Out[4]:

	Age	Billing Amount	Room Number
count	10000.000000	10000.000000	10000.000000
mean	51.452200	25516.806778	300.082000
std	19.588974	14067.292709	115.806027
min	18.000000	1000.180837	101.000000
25%	35.000000	13506.523967	199.000000
50%	52.000000	25258.112566	299.000000
75%	68.000000	37733.913727	400.000000
max	85.000000	49995.902283	500.000000

In [5]:
```
df['Medical Condition'].unique()
```

Out[5]:
```
array(['Diabetes', 'Asthma', 'Obesity', 'Arthritis', 'Hypertension',
       'Cancer'], dtype=object)
```

In [6]:
```
df["Blood Type"].unique()
```

Out[6]:
```
array(['O-', 'O+', 'B-', 'AB+', 'A+', 'AB-', 'A-', 'B+'], dtype=object)
```

In [7]:
```
df["Gender"].unique()
```

Out[7]:
```
array(['Female', 'Male'], dtype=object)
```

In [8]:
```
df["Medication"].unique()
```

Out[8]:
```
array(['Aspirin', 'Lipitor', 'Penicillin', 'Paracetamol', 'Ibuprofen'],
      dtype=object)
```

In [9]:
```
df["Test Results"].unique()
```

Out[9]:
```
array(['Inconclusive', 'Normal', 'Abnormal'], dtype=object)
```

In [10]:
```
df["Admission Type"].unique()
```

Out[10]:
```
array(['Elective', 'Emergency', 'Urgent'], dtype=object)
```

In [11]:
```
df["Insurance Provider"].unique()
```

Out[11]:
```
array(['Medicare', 'UnitedHealthcare', 'Aetna', 'Cigna', 'Blue Cross'],
      dtype=object)
```

In [12]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 15 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   Name                10000 non-null  object 
 1   Age                 10000 non-null  int64  
 2   Gender              10000 non-null  object 
 3   Blood Type          10000 non-null  object 
 4   Medical Condition   10000 non-null  object 
 5   Date of Admission   10000 non-null  object 
 6   Doctor              10000 non-null  object 
 7   Hospital            10000 non-null  object 
 8   Insurance Provider  10000 non-null  object 
 9   Billing Amount      10000 non-null  float64
 10  Room Number         10000 non-null  int64  
 11  Admission Type      10000 non-null  object 
 12  Discharge Date      10000 non-null  object 
 13  Medication          10000 non-null  object 
 14  Test Results        10000 non-null  object 
dtypes: float64(1), int64(2), object(12)
memory usage: 1.1+ MB
```

In [13]:
```
df.corr()
```

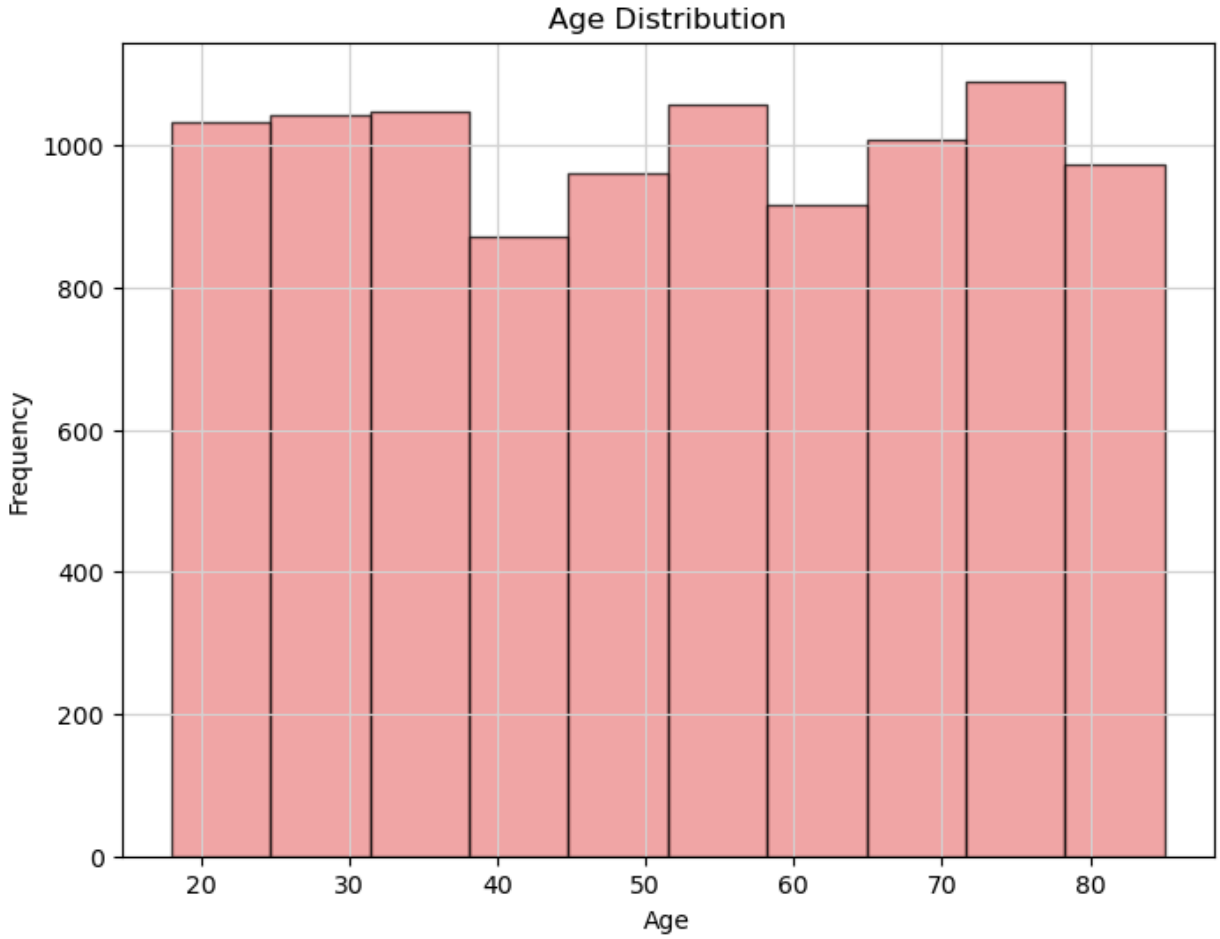
Out[13]:

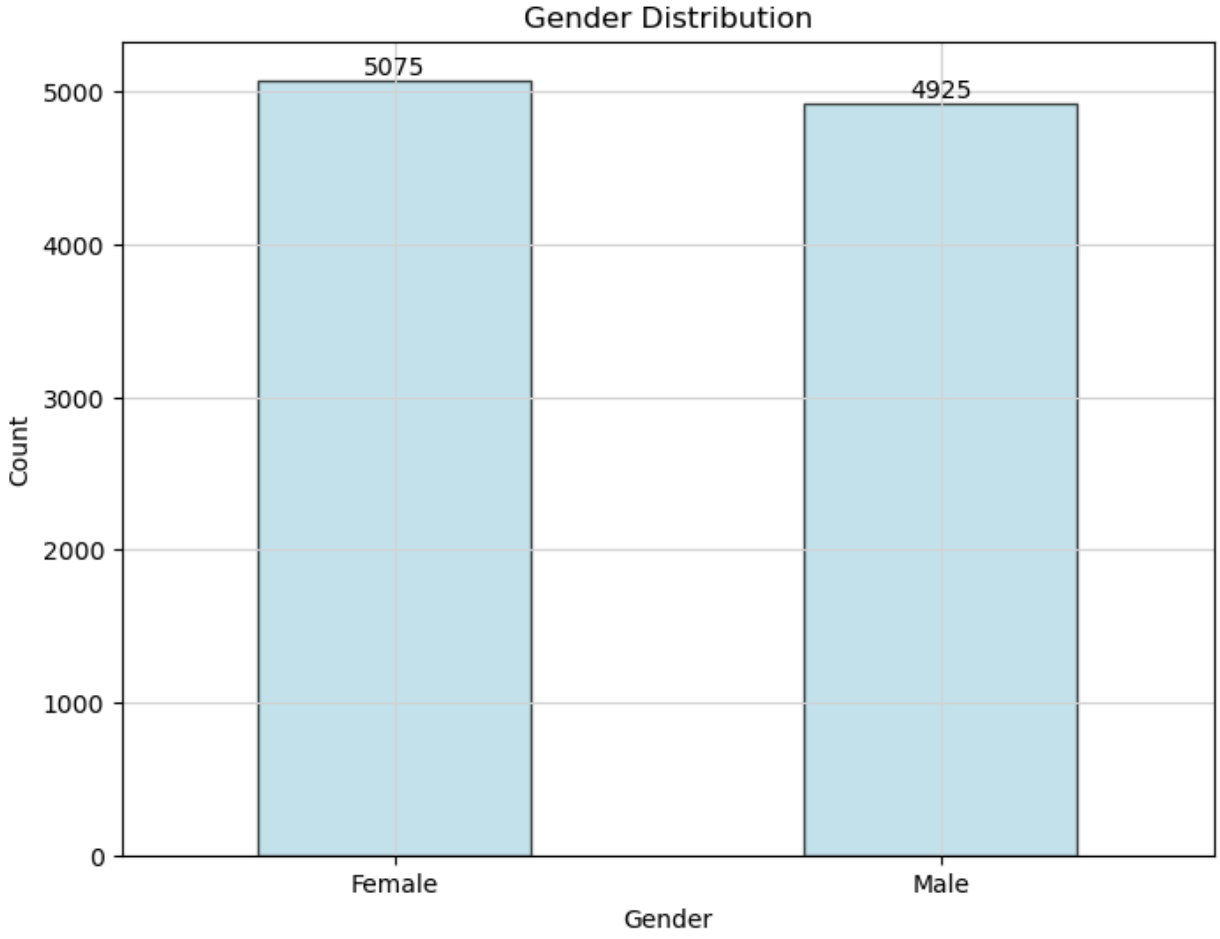
	Age	Billing Amount	Room Number
Age	1.000000	-0.009483	-0.005371
Billing Amount	-0.009483	1.000000	-0.006160
Room Number	-0.005371	-0.006160	1.000000

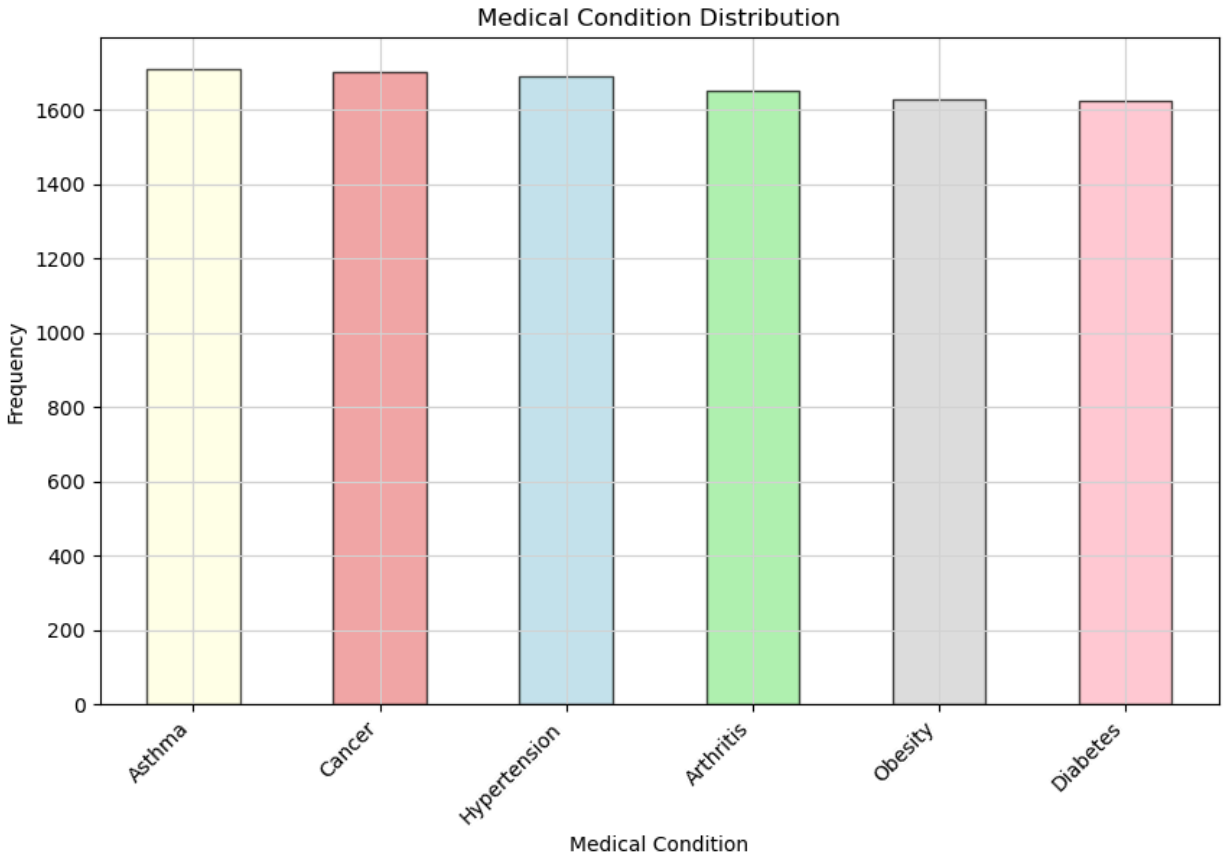
Exploratory Data Analysis

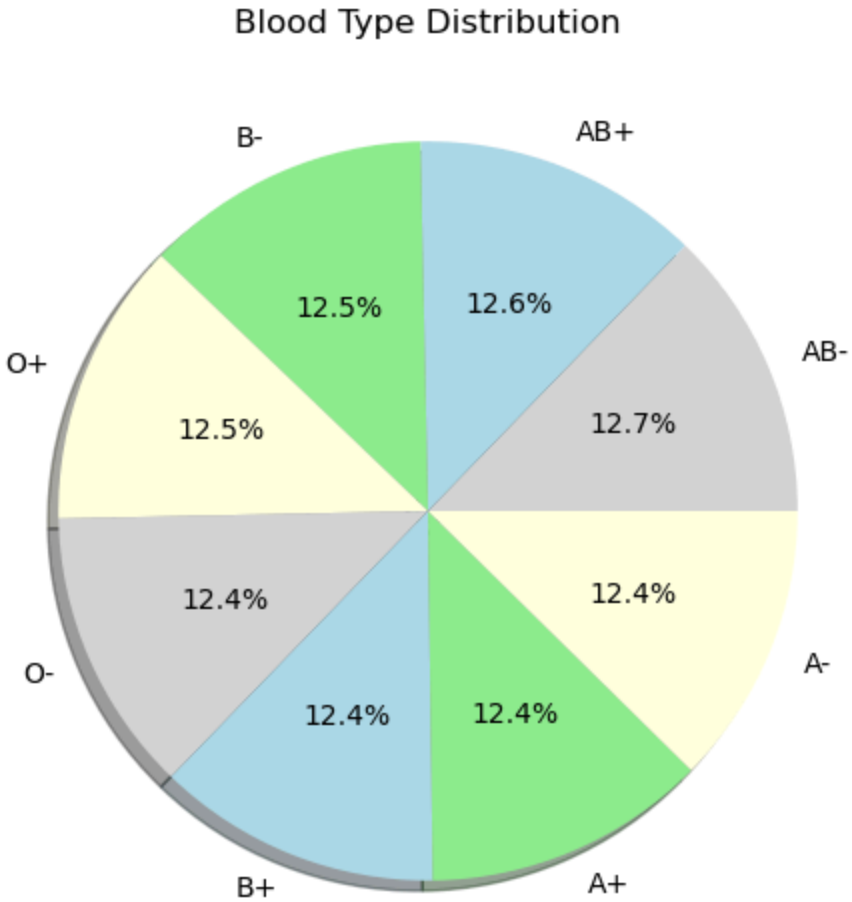
In [14]:
```
# 1. Histogram of Age distribution
plt.figure(figsize=(8, 6))
```

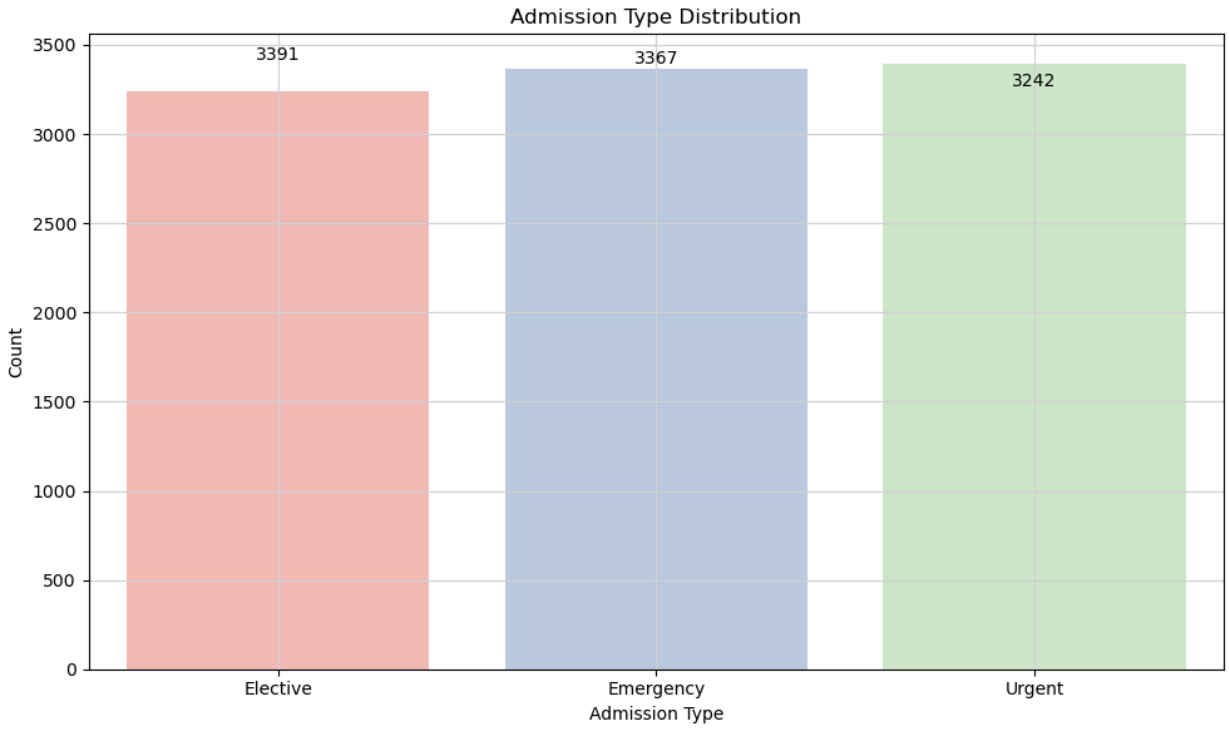
```
plt.hist(df['Age'], bins=10, color='lightcoral', edgecolor='black', alpha=0.7)
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.grid(color='lightgrey')
plt.show()

# 2. Bar plot of Gender distribution
plt.figure(figsize=(8, 6))
gender_counts = df['Gender'].value_counts()
bars = gender_counts.plot(kind='bar', color='lightblue', edgecolor='black', al
plt.title('Gender Distribution')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.grid(color='lightgrey')
for i, v in enumerate(gender_counts):
    plt.text(i, v + 5, str(v), ha='center', va='bottom')

plt.show()


# 3. Bar plot of Medical Condition distribution
plt.figure(figsize=(10, 6))
colors = ['lightyellow', 'lightcoral', 'lightblue', 'lightgreen', 'lightgrey',
medical_condition_counts = df['Medical Condition'].value_counts()
medical_condition_counts_sorted = medical_condition_counts.sort_values(ascendi

medical_condition_counts_sorted.plot(kind='bar', color=colors, edgecolor='blac
plt.title('Medical Condition Distribution')
plt.xlabel('Medical Condition')
plt.ylabel('Frequency')
plt.xticks(rotation=45, ha='right')
plt.grid(color='lightgrey')
plt.show()

# 4. Pie chart of Blood Type distribution
plt.figure(figsize=(8, 6))
df['Blood Type'].value_counts().plot(kind='pie', autopct='%1.1f%%', colors=['l
plt.title('Blood Type Distribution')
plt.ylabel('')
plt.show()


# 5. Count plot for 'Admission Type'
plt.figure(figsize=(10, 6))
admission_type_counts = df['Admission Type'].value_counts()
sns.countplot(x='Admission Type', data=df, palette='Pastel1')

# Add data point values on top of each bar
for i, count in enumerate(admission_type_counts):
    plt.text(i, count + 5, str(count), ha='center', va='bottom')

plt.title('Admission Type Distribution')
plt.xlabel('Admission Type')
plt.ylabel('Count')
plt.grid(color='lightgrey')

plt.tight_layout()
plt.show()
```


Age Distribution
Frequency
0
200
400
600
800
1000
20
30
40
50
60
70
80
Age


Gender Distribution
Count
0
1000
2000
3000
4000
5000
5075
4925
Female
Male
Gender


Medical Condition Distribution
Frequency
0
200
400
600
800
1000
1200
1400
1600
Asthma
Cancer
Hypertension
Arthritis
Obesity
Diabetes
Medical Condition


Blood Type Distribution
B-
AB+
O+
AB-
O-
A-
B+
A+
12.5%
12.6%
12.5%
12.7%
12.4%
12.4%
12.4%
12.4%



Preprocessing the Data

Dropping the insignificant columns

In [15]:
```
df2 = df.drop(columns = ["Name","Date of Admission", "Doctor","Hospital" , "Dis
df2.head()
```

Out[15]:

	Age	Gender	Blood Type	Medical Condition	Insurance Provider	Billing Amount	Admission Type	Medication	
0	81	Female	O-	Diabetes	Medicare	37490.983364	Elective	Aspirin	Inc
1	35	Male	O+	Asthma	UnitedHealthcare	47304.064845	Emergency	Lipitor	
2	61	Male	B-	Obesity	Medicare	36874.896997	Emergency	Lipitor	
3	49	Male	B-	Asthma	Medicare	23303.322092	Urgent	Penicillin	
4	51	Male	O-	Arthritis	UnitedHealthcare	18086.344184	Urgent	Paracetamol	

In [16]:
```
df2.describe()
```

Out[16]:

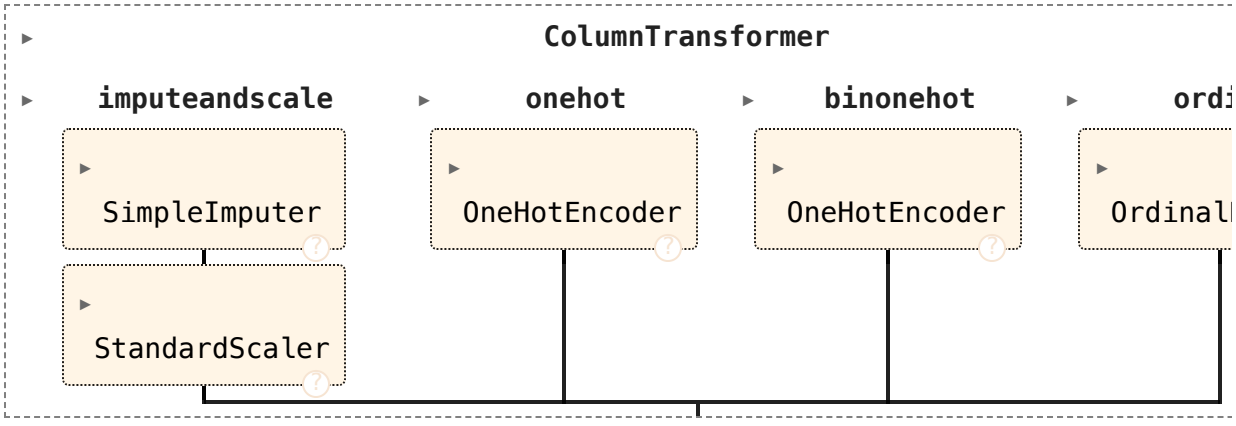
	Age	Billing Amount
count	10000.000000	10000.000000
mean	51.452200	25516.806778
std	19.588974	14067.292709
min	18.000000	1000.180837
25%	35.000000	13506.523967
50%	52.000000	25258.112566
75%	68.000000	37733.913727
max	85.000000	49995.902283

In [17]:
```
numeric_feats = ["Age", "Billing Amount"]
categorical_feats = ["Medical Condition", "Blood Type", "Medication", "Test Re
binary_feats = ["Gender"]
ordinal_feats = ["Admission Type"]
```

In [18]:
```
ct = ColumnTransformer(
    [
        ("imputeandscale", make_pipeline(SimpleImputer(), StandardScaler()), nu
        ("onehot", OneHotEncoder(sparse_output=False), categorical_feats),
        ("binonehot", OneHotEncoder(sparse_output=False, drop="if_binary"), bi
        ("ordinal", OrdinalEncoder(categories=[["Elective", "Urgent", "Emergen
    ]
)
```

In [19]:
```
ct
```

Out[19]:



In [20]:
```
df2_transformed = ct.fit_transform(df2)

df2_transformed
```

Out[20]:
```
array([[ 1.50846482,  0.85124946,  0.        , ...,  0.        ,
         0.        ,  0.        ],
       [-0.83991244,  1.54886572,  0.        , ...,  1.        ,
         1.        ,  2.        ],
       [ 0.48743123,  0.80745161,  0.        , ...,  0.        ,
         1.        ,  2.        ],
       ...,
       [ 0.13006947,  1.70918448,  1.        , ...,  0.        ,
         1.        ,  0.        ],
       [ 1.66161985, -0.01993817,  1.        , ...,  1.        ,
         1.        ,  1.        ],
       [-1.60568763,  0.83226707,  1.        , ...,  0.        ,
         1.        ,  2.        ]])
```

In [21]:
```
column_names = (
    numeric_feats
    + ct.named_transformers_["onehot"].get_feature_names_out().tolist()
    + binary_feats
    + ordinal_feats
)
column_names
```

Out[21]:
```
['Age',
 'Billing Amount',
 'Medical Condition_Arthritis',
 'Medical Condition_Asthma',
 'Medical Condition_Cancer',
 'Medical Condition_Diabetes',
 'Medical Condition_Hypertension',
 'Medical Condition_Obesity',
 'Blood Type_A+',
 'Blood Type_A-',
 'Blood Type_AB+',
 'Blood Type_AB-',
 'Blood Type_B+',
 'Blood Type_B-',
 'Blood Type_O+',
 'Blood Type_O-',
 'Medication_Aspirin',
 'Medication_Ibuprofen',
 'Medication_Lipitor',
 'Medication_Paracetamol',
 'Medication_Penicillin',
 'Test Results_Abnormal',
 'Test Results_Inconclusive',
 'Test Results_Normal',
 'Insurance Provider_Aetna',
 'Insurance Provider_Blue Cross',
 'Insurance Provider_Cigna',
 'Insurance Provider_Medicare',
 'Insurance Provider_UnitedHealthcare',
 'Gender',
 'Admission Type']
```

In [22]:
```
ct.named_transformers_
```

Out[22]:
```
{'imputeandscale': Pipeline(steps=[('simpleimputer', SimpleImputer()),
                                   ('standardscaler', StandardScaler())]),
 'onehot': OneHotEncoder(sparse_output=False),
 'binonehot': OneHotEncoder(drop='if_binary', sparse_output=False),
 'ordinal': OrdinalEncoder(categories=[['Elective', 'Urgent', 'Emergency']])}
```

In [23]:
```
df3 = pd.DataFrame(df2_transformed, columns= column_names)

df3.head()
```

Out[23]:

	Age	Billing Amount	Medical Condition_Arthritis	Medical Condition_Asthma	Medical Condition_Cancer	I Condition_D
0	1.508465	0.851249	0.0	0.0	0.0	
1	-0.839912	1.548866	0.0	1.0	0.0	
2	0.487431	0.807452	0.0	0.0	0.0	
3	-0.125189	-0.157358	0.0	1.0	0.0	
4	-0.023086	-0.528235	1.0	0.0	0.0	

5 rows × 31 columns

Selecting the Target Variable and performing Test - train split

Target : Test Results_Abnormal

In [24]:
```
X = df3.drop(columns=["Test Results_Abnormal"])


Y = df3["Test Results_Abnormal"]


X

Y
```

Out[24]:
```
0       0.0
1       0.0
2       0.0
3       1.0
4       0.0
       ...
9995    1.0
9996    0.0
9997    0.0
9998    0.0
9999    1.0
Name: Test Results_Abnormal, Length: 10000, dtype: float64
```

In [25]:
```
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.2, rando

shape_dict = {
```

```
    "Data portion": ["X", "y", "X_train", "y_train", "X_test", "y_test"],
    "Shape": [
        X.shape,
        Y.shape,
        x_train.shape,
        y_train.shape,
        x_test.shape,
        y_test.shape,
    ],
}
print("Target: Test Results_Abnormal")

shape_df = pd.DataFrame(shape_dict)
shape_df
```

```
Target: Test Results_Abnormal
```

Out[25]:

	Data portion	Shape
0	X	(10000, 30)
1	y	(10000,)
2	X_train	(8000, 30)
3	y_train	(8000,)
4	X_test	(2000, 30)
5	y_test	(2000,)

Model: K-Nearest Neighbours

Target : Test Results_Abnormal

In [26]:

```
target = "Test Results_Abnormal"
model = KNeighborsClassifier(n_neighbors=5)

model.fit(x_train, y_train)

y_pred = model.predict(x_test)

accuracy = accuracy_score(y_test, y_pred)
print("\Model: K-Nearest Neighbour")
print(f"Accuracy of {target} is : ", accuracy*100,"%")
```

```
\Model: K-Nearest Neighbour
Accuracy of Test Results_Abnormal is :  90.05 %
```

In [27]:

```
# Cross Validation : Model : Random Forest Classifier

cv_score = cross_val_score(model, x_train, y_train, cv=2)

print("\nTarget :  Test Results_Abnormal | Model: K-Nearest Neighbour")
cv_score.mean()
```

```
Target :  Test Results_Abnormal | Model: K-Nearest Neighbour
```

Out[27]:

```
0.866125
```

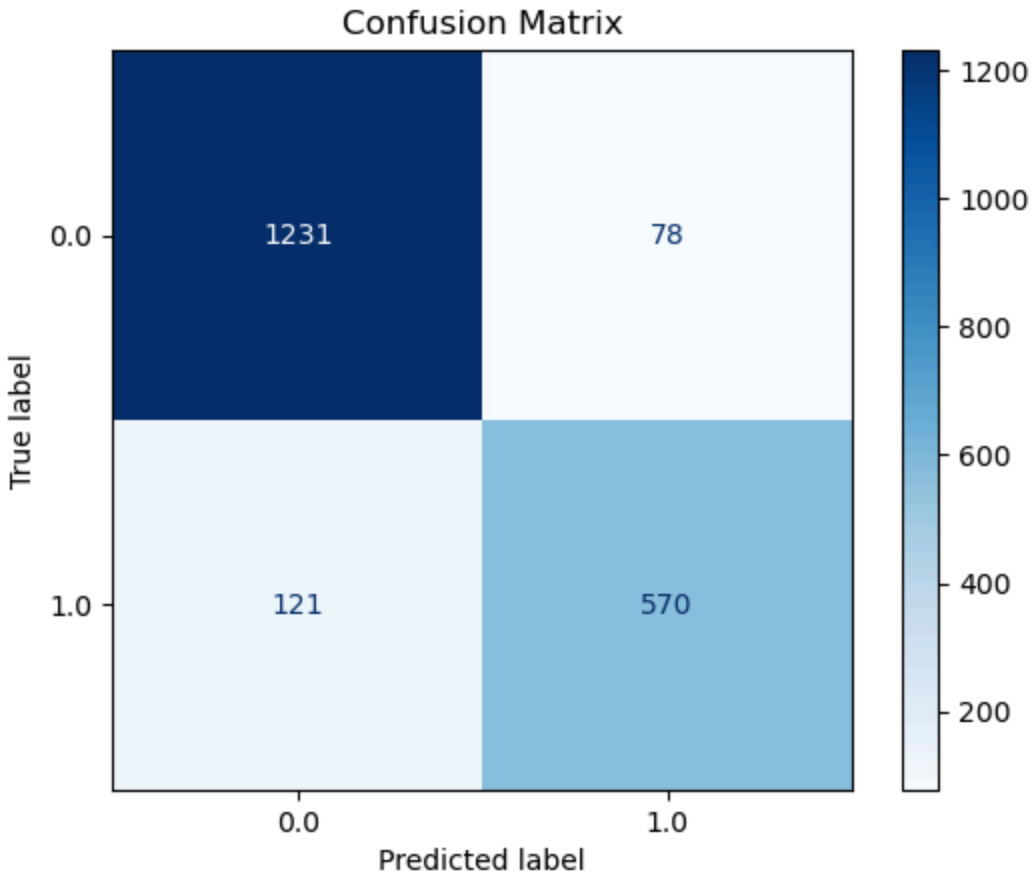
In [28]:
```
#Model : Random Forest Classifier

scores = cross_validate(model, x_train, y_train, cv=10, return_train_score=Tru
pd.DataFrame(scores)
```

Out[28]:

	fit_time	score_time	test_score	train_score
0	0.001972	0.021759	0.90500	0.952083
1	0.001783	0.020089	0.90875	0.948750
2	0.001796	0.018027	0.89250	0.950417
3	0.001761	0.018483	0.90500	0.951389
4	0.001714	0.018900	0.89750	0.950556
5	0.001612	0.018192	0.88500	0.950694
6	0.001620	0.017331	0.88500	0.953750
7	0.001778	0.021980	0.89250	0.952500
8	0.001662	0.017797	0.87375	0.953611
9	0.001637	0.020349	0.89750	0.953611

In [29]:
```
print("\nTarget :  Test Results_Abnormal | Model: K-Nearest Neighbour")
print("---------------------------------------------------------------")

cm = ConfusionMatrixDisplay.from_estimator(model, x_test, y_test, values_forma
plt.title('Confusion Matrix')
plt.show()
```

```
Target :  Test Results_Abnormal | Model: K-Nearest Neighbour
---------------------------------------------------------------
```

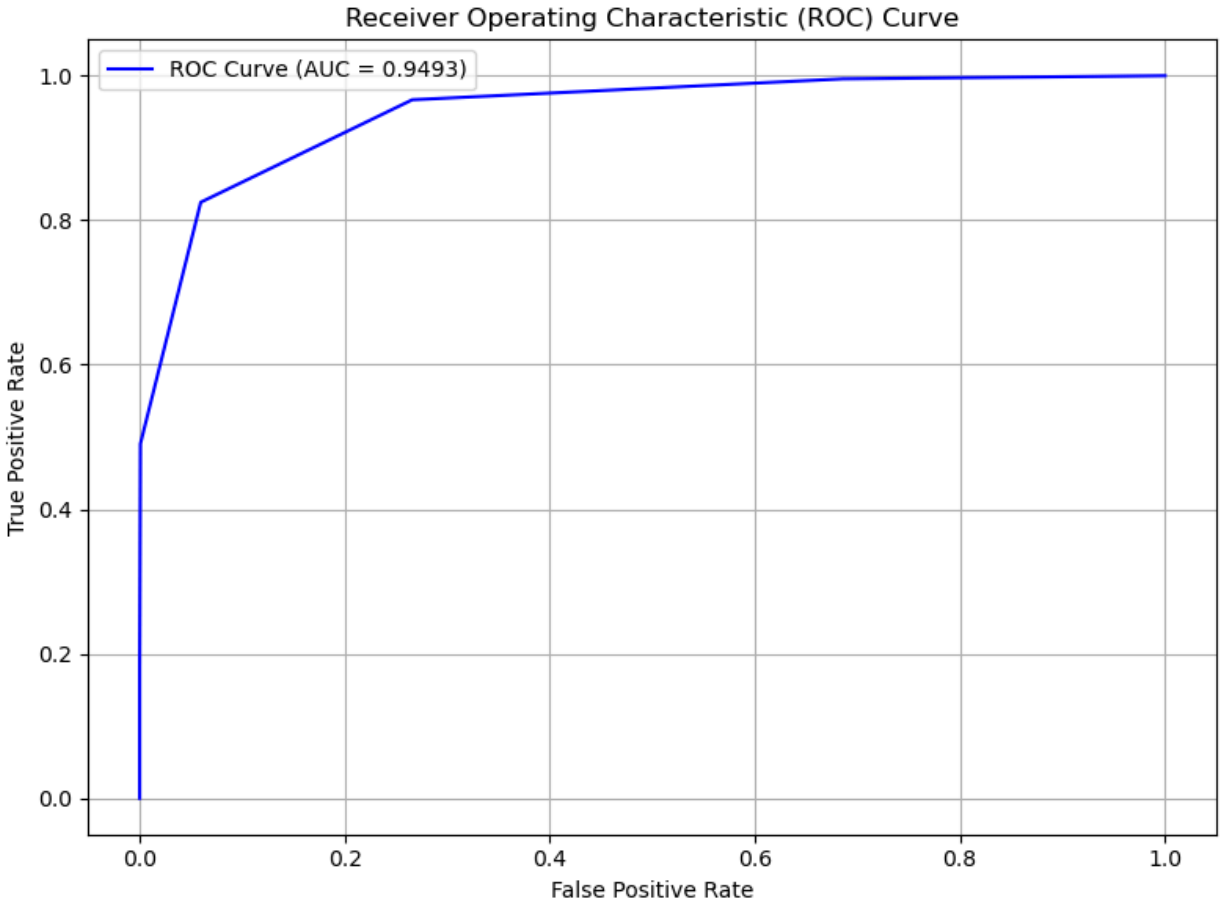


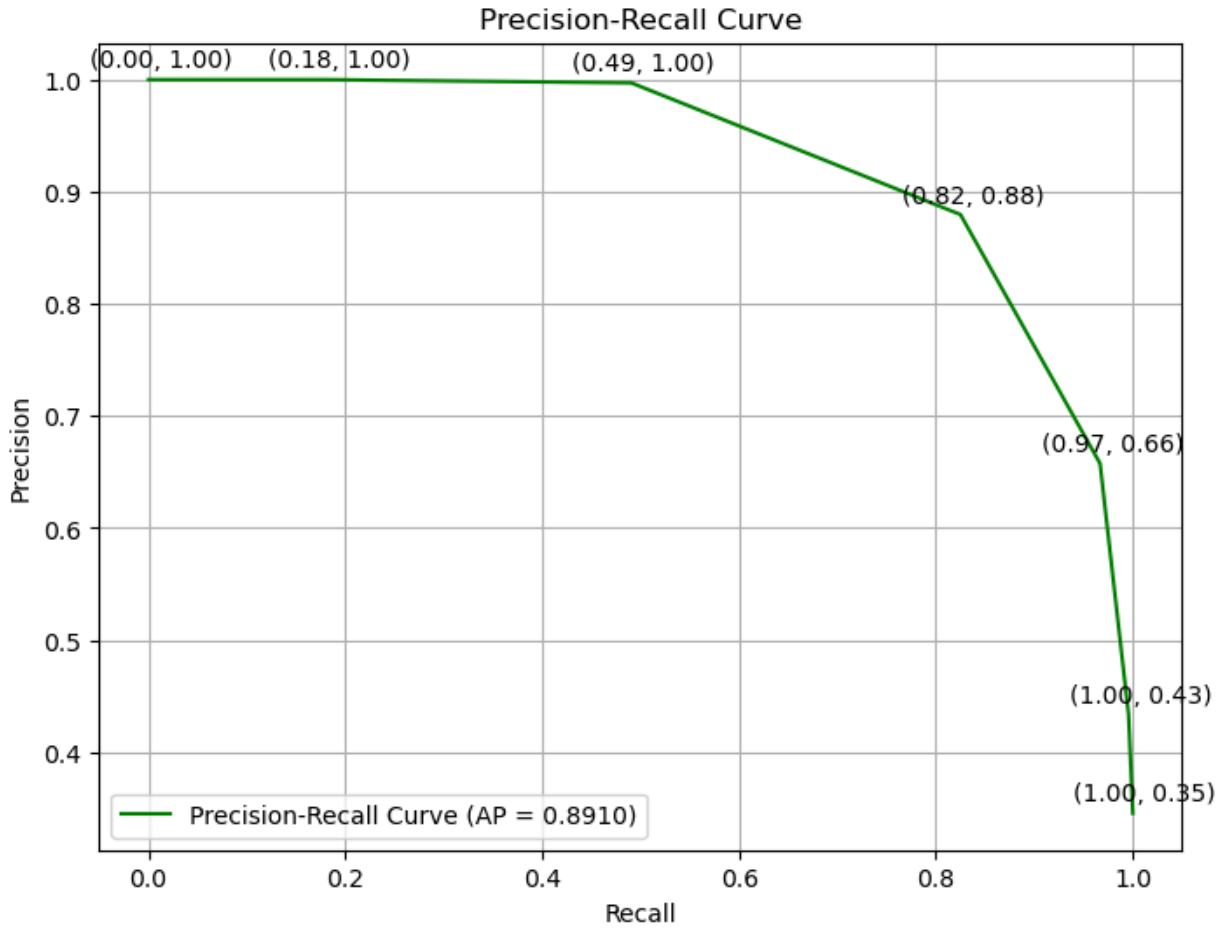
In [30]:
```
print("Target :  Test Results_Abnormal | Model: K-Nearest Neighbour")
print("-----------------------------------------------------------------")

print(
    classification_report(
        y_test, model.predict(x_test), target_names=["No", "Yes"], digits=6
    )
)

y_pred = model.predict(x_test)
rmse = mean_squared_error(y_test, y_pred, squared=False)
mae = mean_absolute_error(y_test, y_pred)
```

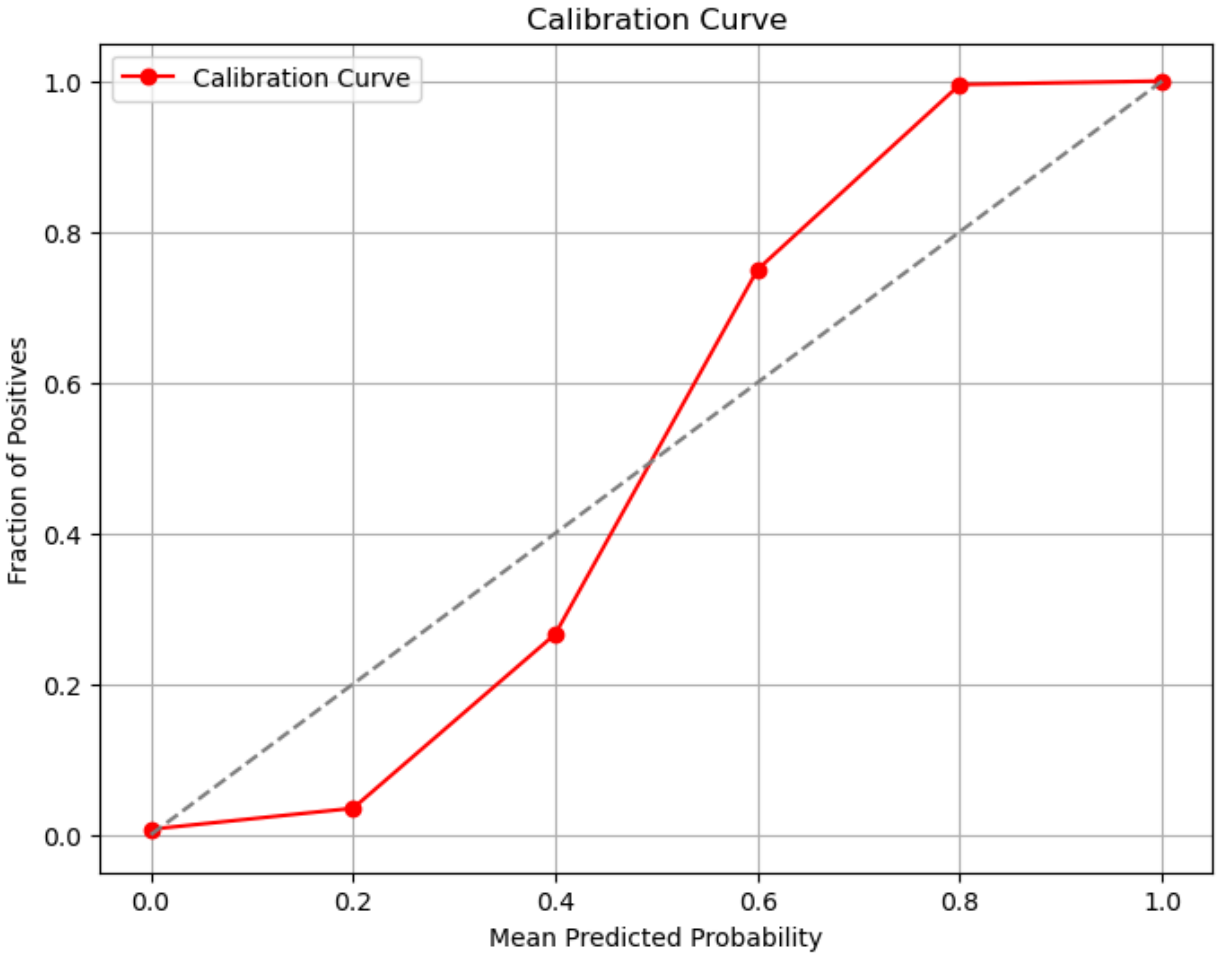
```
Target :  Test Results_Abnormal | Model: K-Nearest Neighbour
-----------------------------------------------------------------
              precision    recall  f1-score   support

          No   0.910503  0.940413  0.925216      1309
         Yes   0.879630  0.824891  0.851382       691

    accuracy                       0.900500      2000
   macro avg   0.895066  0.882652  0.888299      2000
weighted avg   0.899836  0.900500  0.899706      2000
```

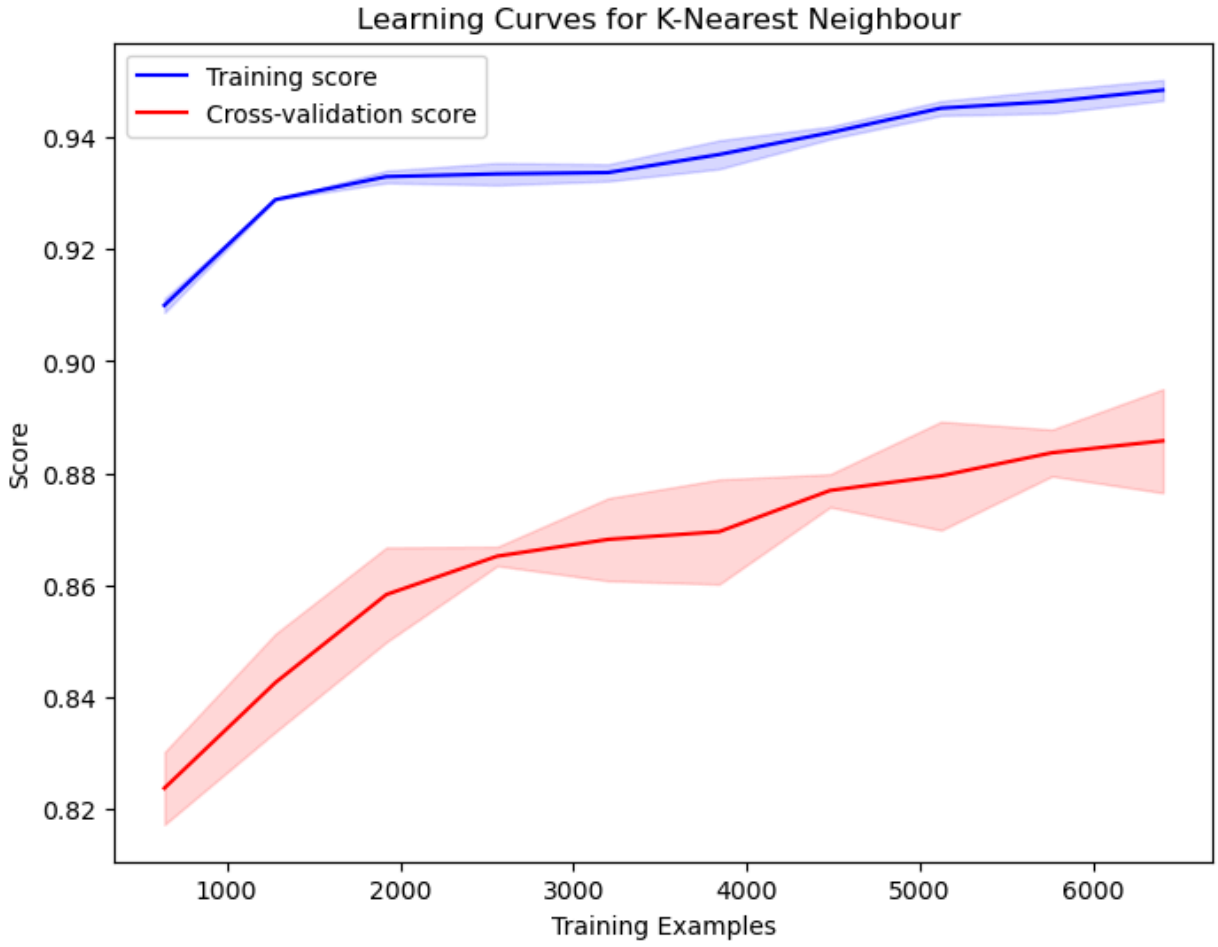
In [31]:
```
fpr, tpr, thresholds = roc_curve(y_test, model.predict_proba(x_test)[:, 1])
auc_score = roc_auc_score(y_test, model.predict_proba(x_test)[:, 1])

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {auc_score:.4f})', color='blue')
plt.xlabel('False Positive Rate')
```

```
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# Generate Precision-Recall Curve and calculate average precision score
precision, recall, thresholds = precision_recall_curve(y_test, model.predict_p
ap_score = average_precision_score(y_test, model.predict_proba(x_test)[:, 1])

# Calculate F-beta score
beta = 2
f_beta_score = (1 + beta**2) * (precision * recall) / ((beta**2 * precision) +

# Annotate data point values on Precision-Recall Curve
plt.figure(figsize=(8, 6))
plt.plot(recall, precision, label=f'Precision-Recall Curve (AP = {ap_score:.4f}
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.legend()
plt.grid(True)

# Annotate data point values
for i in range(len(precision)):
    plt.annotate(f'({recall[i]:.2f}, {precision[i]:.2f})', (recall[i], precisi

plt.show()


# Calculate Calibration Curve
prob_true, prob_pred = calibration_curve(y_test, model.predict_proba(x_test)[:

plt.figure(figsize=(8, 6))
plt.plot(prob_pred, prob_true, marker='o', label='Calibration Curve', color='r
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel('Mean Predicted Probability')
plt.ylabel('Fraction of Positives')
plt.title('Calibration Curve')
plt.legend()
plt.grid(True)
plt.show()

train_sizes, train_scores, test_scores = learning_curve(model, x_train, y_trai
train_mean = np.mean(train_scores, axis=1)
train_std = np.std(train_scores, axis=1)
test_mean = np.mean(test_scores, axis=1)
test_std = np.std(test_scores, axis=1)
plt.figure(figsize=(8, 6))
plt.plot(train_sizes, train_mean, label='Training score', color='b')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, a
plt.plot(train_sizes, test_mean, label='Cross-validation score', color='r')
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha
plt.title("Learning Curves for K-Nearest Neighbour ")
plt.xlabel("Training Examples")
plt.ylabel("Score")
plt.legend(loc="best")
plt.show()
```


Receiver Operating Characteristic (ROC) Curve
ROC Curve (AUC = 0.9493)
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0


Precision-Recall Curve
(0.00, 1.00)
(0.18, 1.00)
(0.49, 1.00)
(0.82, 0.88)
(0.97, 0.66)
(1.00, 0.43)
(1.00, 0.35)
Precision-Recall Curve (AP = 0.8910)
Precision
Recall
0.4
0.5
0.6
0.7
0.8
0.9
1.0


Calibration Curve
Calibration Curve
Fraction of Positives
Mean Predicted Probability
0.0
0.2
0.4
0.6
0.8
1.0


Learning Curves for K-Nearest Neighbour
Training score
Cross-validation score
Score
Training Examples
0.82
0.84
0.86
0.88
0.90
0.92
0.94
1000
2000
3000
4000
5000
6000

Model: Random Forest Classifier

Target : Test Results_Abnormal

In [32]:
```
target = "Test Results_Abnormal"
model = RandomForestClassifier(n_estimators=2, random_state=42)

model.fit(x_train, y_train)

y_pred = model.predict(x_test)

accuracy = accuracy_score(y_test, y_pred)
print("\nModel: Random Forest Classifier\n")
print(f"Accuracy of Target: {target} is : ", accuracy*100,"%")
```

```
Model: Random Forest Classifier

Accuracy of Target: Test Results_Abnormal is :  92.25 %
```

In [33]:
```
# Cross Validation : Model : Random Forest Classifier
cv_score = cross_val_score(model, x_train, y_train, cv=2)

print("\nTarget :  Test Results_Abnormal | Model: Random Forest Classifier\n")
cv_score.mean()
```

```
Target :  Test Results_Abnormal | Model: Random Forest Classifier
```

Out[33]:
```
0.883875
```

In [34]:
```
print("\nTarget :  Test Results_Abnormal | Model: Random Forest Classifier")

scores = cross_validate(model, x_train, y_train, cv=10, return_train_score=Tru
pd.DataFrame(scores)
```

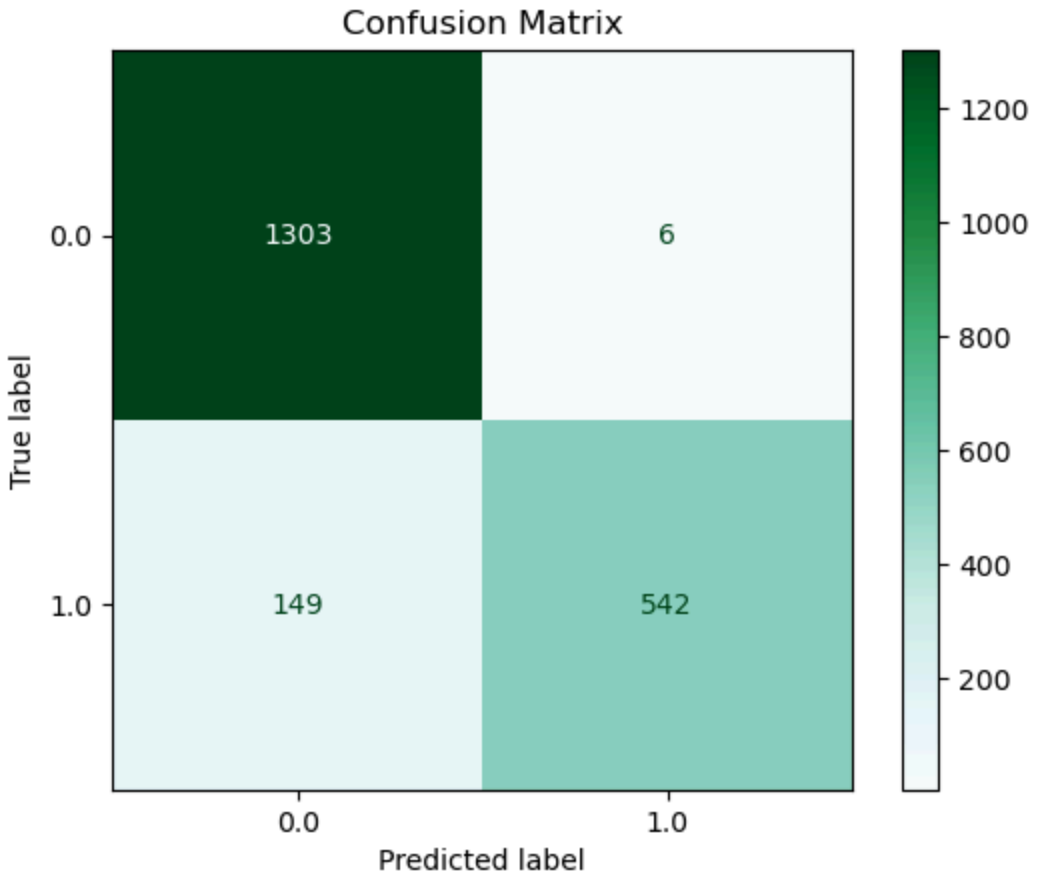
```
Target :  Test Results_Abnormal | Model: Random Forest Classifier
```

Out[34]:

	fit_time	score_time	test_score	train_score
0	0.020041	0.001862	0.90125	0.968889
1	0.015586	0.001509	0.91625	0.965417
2	0.015899	0.001589	0.90250	0.963611
3	0.013637	0.001547	0.90625	0.961111
4	0.014163	0.001678	0.90750	0.975278
5	0.014237	0.001487	0.88500	0.953750
6	0.013183	0.001863	0.91000	0.969583
7	0.013647	0.001558	0.91375	0.961111
8	0.014214	0.001571	0.89000	0.961111
9	0.013513	0.001490	0.91125	0.965694

In [35]:
```
print("\nConfusion Matrix : Target :  Test Results_Abnormal | Model: Random Fo
print("----------------------------------------------------------------------")

cm = ConfusionMatrixDisplay.from_estimator(model, x_test, y_test, values_forma
plt.title("Confusion Matrix")
plt.show()
```

```
Confusion Matrix : Target :  Test Results_Abnormal | Model: Random Forest Clas
sifier

----------------------------------------------------------------------
```



In [36]:
```
print("\nTarget :  Test Results_Abnormal | Model: Random Forest Classifier\n")
print("----------------------------------------------------------------------")

print(
    classification_report(
        y_test, model.predict(x_test), target_names=["No", "Yes"], digits=6
    )
)
```

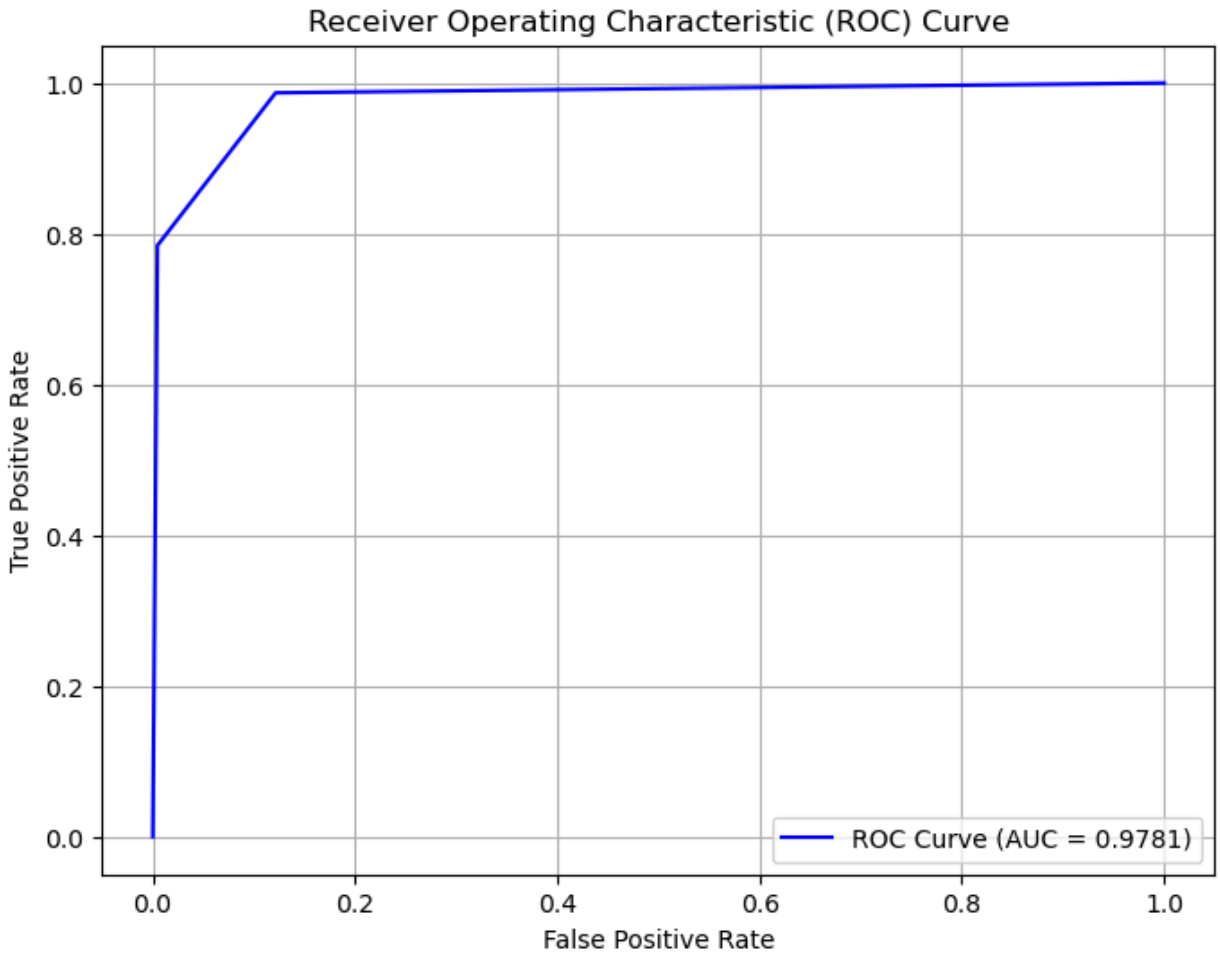
```
Target :  Test Results_Abnormal | Model: Random Forest Classifier

------------------------------------------------------------
              precision    recall  f1-score   support

          No   0.897383  0.995416  0.943861      1309
         Yes   0.989051  0.784370  0.874899       691

    accuracy                       0.922500      2000
   macro avg   0.943217  0.889893  0.909380      2000
weighted avg   0.929054  0.922500  0.920035      2000
```

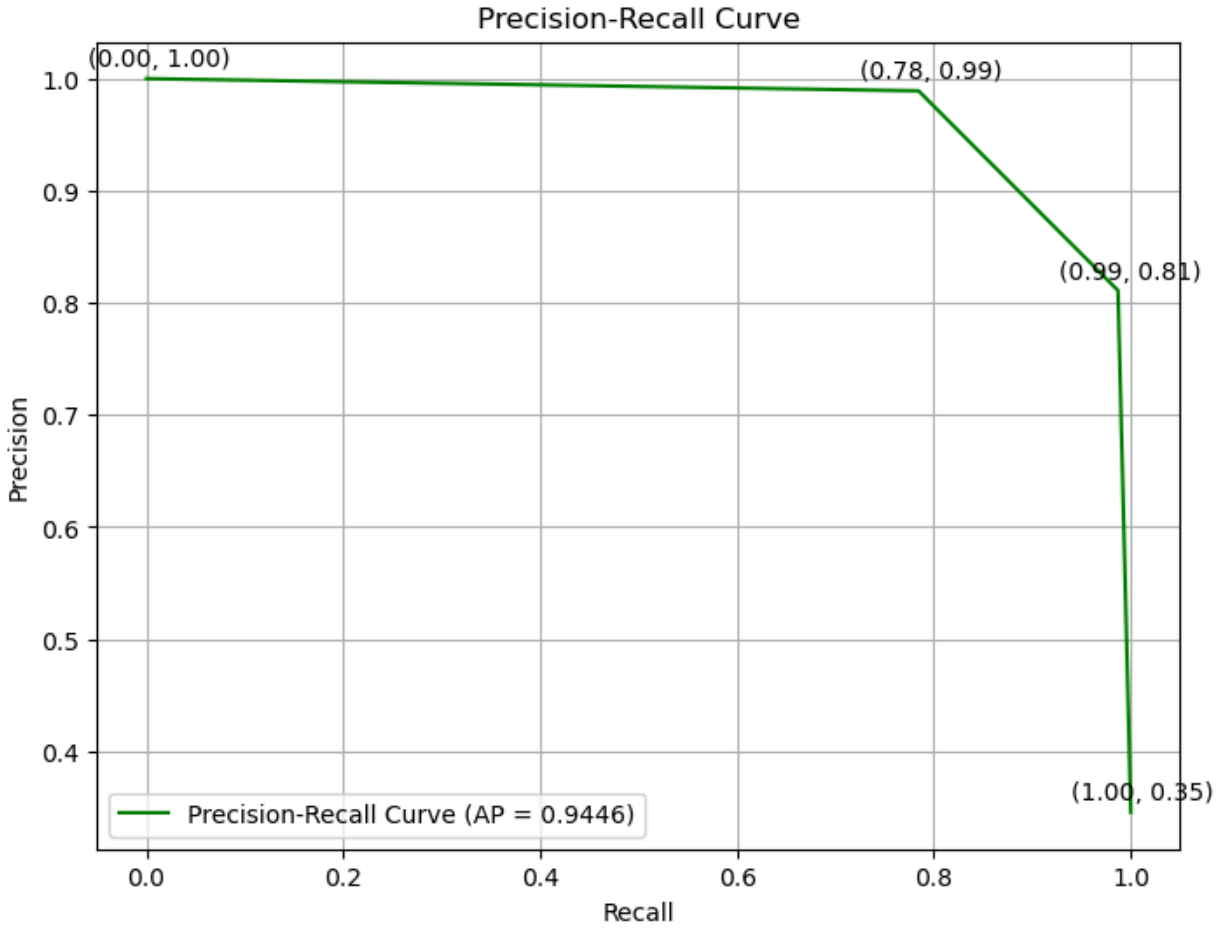
In [37]:

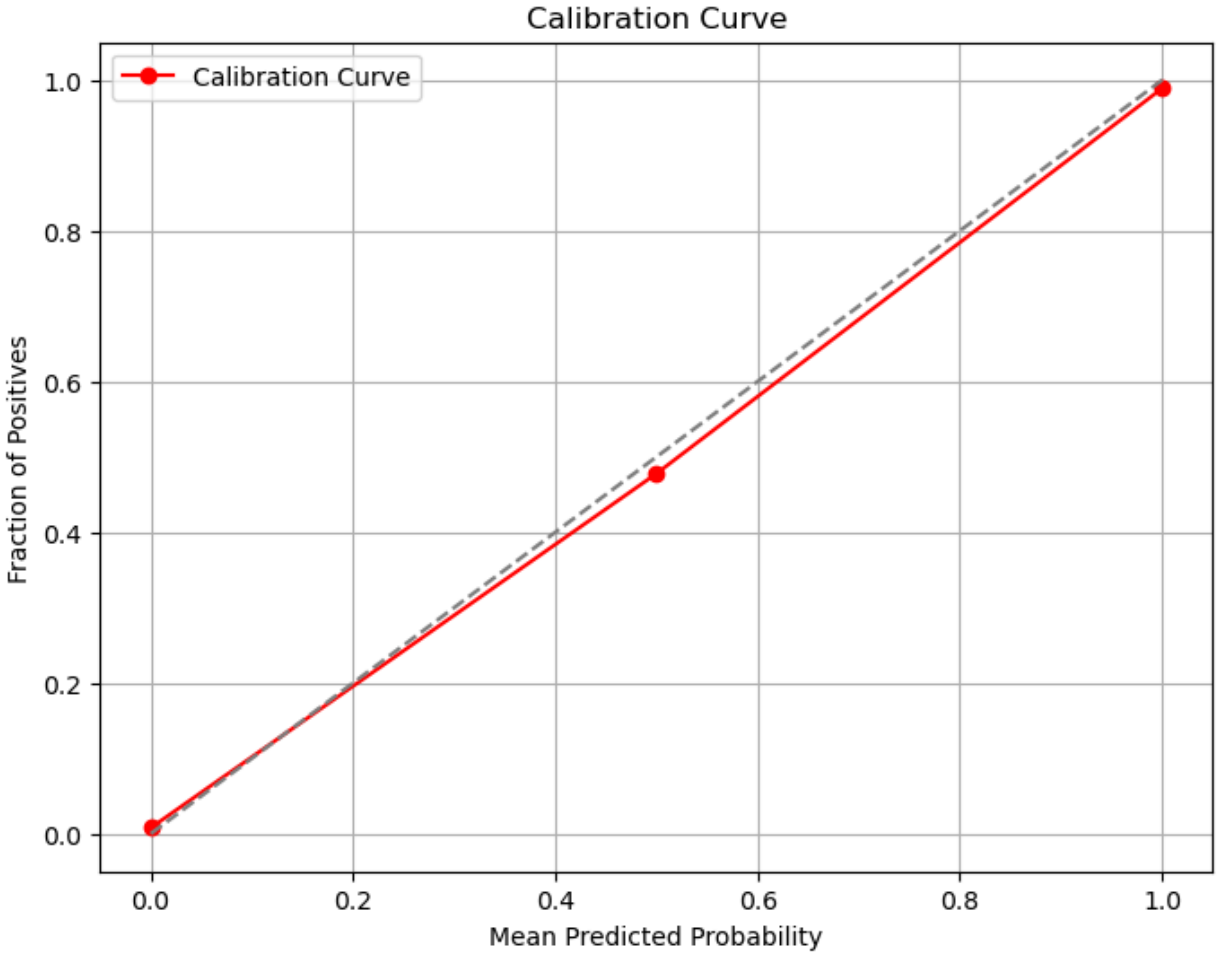
```
print("\nTarget :  Test Results_Abnormal | Model: Random Forest Classifier\n")
print("------------------------------------------------------------")


# Generate ROC Curve and calculate AUC score
fpr, tpr, thresholds = roc_curve(y_test, model.predict_proba(x_test)[:, 1])
auc_score = roc_auc_score(y_test, model.predict_proba(x_test)[:, 1])

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {auc_score:.4f})', color='blue')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend()
plt.grid(True)
plt.show()

# Generate Precision-Recall Curve and calculate average precision score
precision, recall, thresholds = precision_recall_curve(y_test, model.predict_p
ap_score = average_precision_score(y_test, model.predict_proba(x_test)[:, 1])

# Calculate F-beta score
beta = 2
f_beta_score = (1 + beta**2) * (precision * recall) / ((beta**2 * precision) +

# Annotate data point values on Precision-Recall Curve
plt.figure(figsize=(8, 6))
plt.plot(recall, precision, label=f'Precision-Recall Curve (AP = {ap_score:.4f
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.legend()
plt.grid(True)

# Annotate data point values
for i in range(len(precision)):
    plt.annotate(f'({recall[i]:.2f}, {precision[i]:.2f})', (recall[i], precisi

plt.show()


# Calculate Calibration Curve
prob_true, prob_pred = calibration_curve(y_test, model.predict_proba(x_test)[:

plt.figure(figsize=(8, 6))
plt.plot(prob_pred, prob_true, marker='o', label='Calibration Curve', color='r
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
```

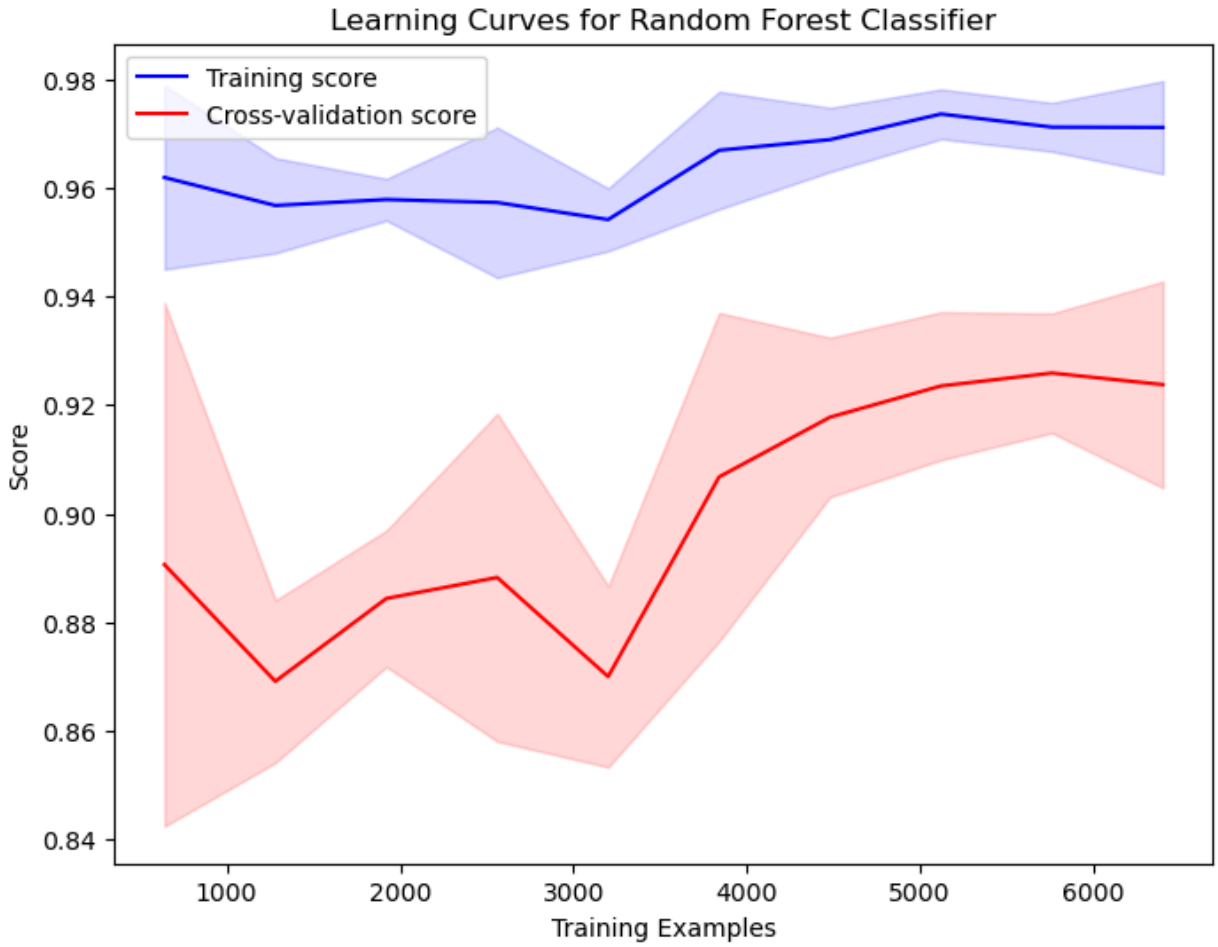
```
plt.xlabel('Mean Predicted Probability')
plt.ylabel('Fraction of Positives')
plt.title('Calibration Curve')
plt.legend()
plt.grid(True)
plt.show()

train_sizes, train_scores, test_scores = learning_curve(model, x_train, y_trai
train_mean = np.mean(train_scores, axis=1)
train_std = np.std(train_scores, axis=1)
test_mean = np.mean(test_scores, axis=1)
test_std = np.std(test_scores, axis=1)
plt.figure(figsize=(8, 6))
plt.plot(train_sizes, train_mean, label='Training score', color='b')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, a
plt.plot(train_sizes, test_mean, label='Cross-validation score', color='r')
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha
plt.title("Learning Curves for Random Forest Classifier")
plt.xlabel("Training Examples")
plt.ylabel("Score")
plt.legend(loc="best")
plt.show()
```

```
Target :  Test Results_Abnormal | Model: Random Forest Classifier

_________________________________________________________________
```




Precision-Recall Curve
(0.00, 1.00)
(0.78, 0.99)
(0.99, 0.81)
(1.00, 0.35)
Precision-Recall Curve (AP = 0.9446)
Precision
Recall
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.0
0.2
0.4
0.6
0.8
1.0


Calibration Curve
Calibration Curve
Fraction of Positives
Mean Predicted Probability
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0



# #

Based on the above evaluations, Random Forest Classifier high probability of correctly classifing the classes.

Hence we will be uing Random Forest Classifier for the features : Test Result*Inconclusive and Test Result* Normal

# #

Target : Test Results_Inconclusive

In [38]:
```
#Case B : Target value: Test Results_Inconclusive

X = df3.drop(columns=["Test Results_Inconclusive"])


Y = df3["Test Results_Inconclusive"]


X

Y
```

Out[38]:
```
0       1.0
1       0.0
2       0.0
3       0.0
4       0.0
       ...
9995    0.0
9996    0.0
9997    0.0
9998    0.0
9999    0.0
Name: Test Results_Inconclusive, Length: 10000, dtype: float64
```

In [39]:
```
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.2, rando

shape_dict = {
    "Data portion": ["X", "y", "X_train", "y_train", "X_test", "y_test"],
    "Shape": [
        X.shape,
        Y.shape,
        x_train.shape,
        y_train.shape,
        x_test.shape,
        y_test.shape,
    ],
}

shape_df = pd.DataFrame(shape_dict)
shape_df
```

Out[39]:

	Data portion	Shape
0	X	(10000, 30)
1	y	(10000,)
2	X_train	(8000, 30)
3	y_train	(8000,)
4	X_test	(2000, 30)
5	y_test	(2000,)

Model: Random Forest Classifier

In [40]:
```
target = "Test Results_Inconclusive"
model = RandomForestClassifier(n_estimators=3, random_state=42)

model.fit(x_train, y_train)

y_pred = model.predict(x_test)

accuracy = accuracy_score(y_test, y_pred)
print("\nModel: Random Forest Classifier ")
print(f"Target: {target}")
print(f"Accuracy of {target} is : ", accuracy*100,"%")
```

```
Model: Random Forest Classifier
Target: Test Results_Inconclusive
Accuracy of Test Results_Inconclusive is :  96.89999999999999 %
```

In [41]:
```
# Cross Validation : Model : Random Forest Classifier

cv_score = cross_val_score(model, x_train, y_train, cv=2)

print("\nTarget :  Test Results_Inconclusive | Model: Random Forest Classifier"
cv_score.mean()
```

```
Target :  Test Results_Inconclusive | Model: Random Forest Classifier
```

Out[41]:
```
0.972125
```

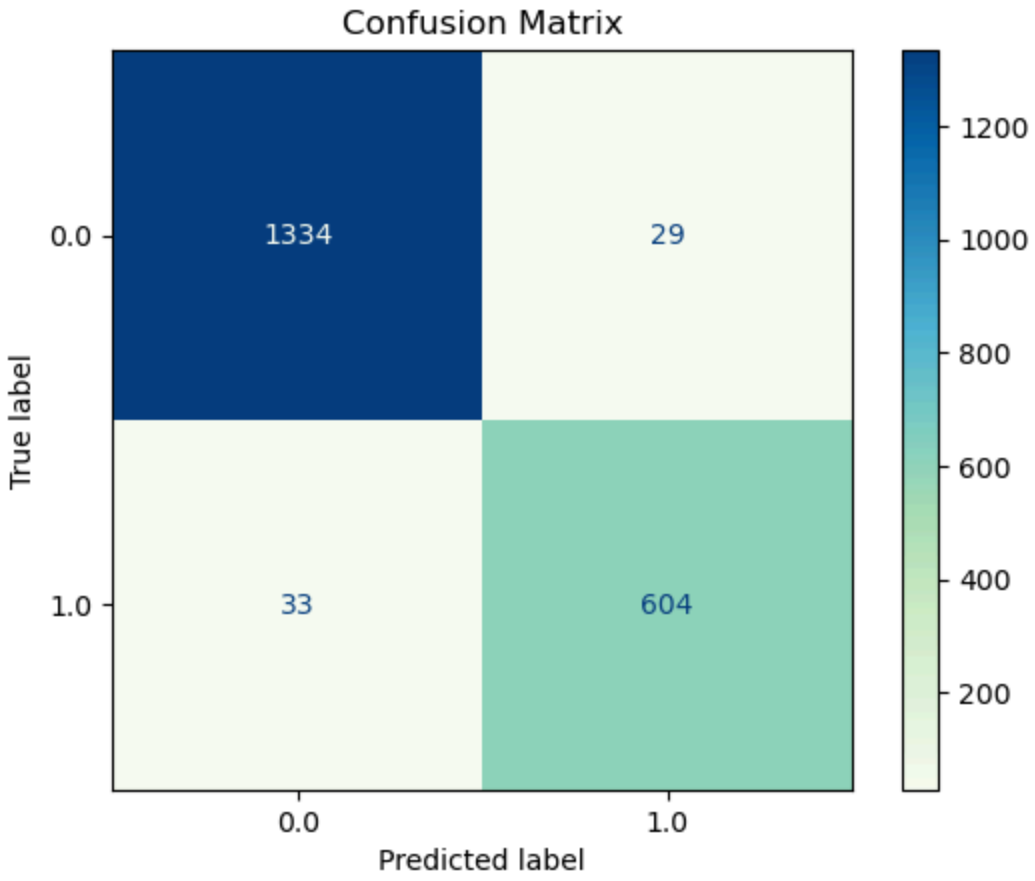
In [42]:
```
#Model : Random Forest Classifier

scores = cross_validate(model, x_train, y_train, cv=10, return_train_score=Tru
pd.DataFrame(scores)
```

Out[42]:

	fit_time	score_time	test_score	train_score
0	0.011901	0.001296	0.99000	0.999306
1	0.013669	0.001378	0.99625	0.999167
2	0.012906	0.001314	0.98875	0.999167
3	0.013431	0.001271	0.98750	0.998750
4	0.012241	0.001318	0.99000	0.998889
5	0.014357	0.001297	0.98875	0.997917
6	0.012463	0.001248	0.99000	0.997361
7	0.012664	0.001231	0.98875	0.999444
8	0.012441	0.001205	0.98500	0.999167
9	0.012815	0.001234	0.98875	0.999306

In [43]:
```
print("\nTarget :  Test Results_Inconclusive | Model: Random Forest Classifier"
print("-------------------------------------------------------------")


from sklearn.metrics import ConfusionMatrixDisplay

cm = ConfusionMatrixDisplay.from_estimator(model, x_test, y_test, values_forma
plt.title("Confusion Matrix")
plt.show()
```

```
Target :  Test Results_Inconclusive | Model: Random Forest Classifier
-------------------------------------------------------------
```

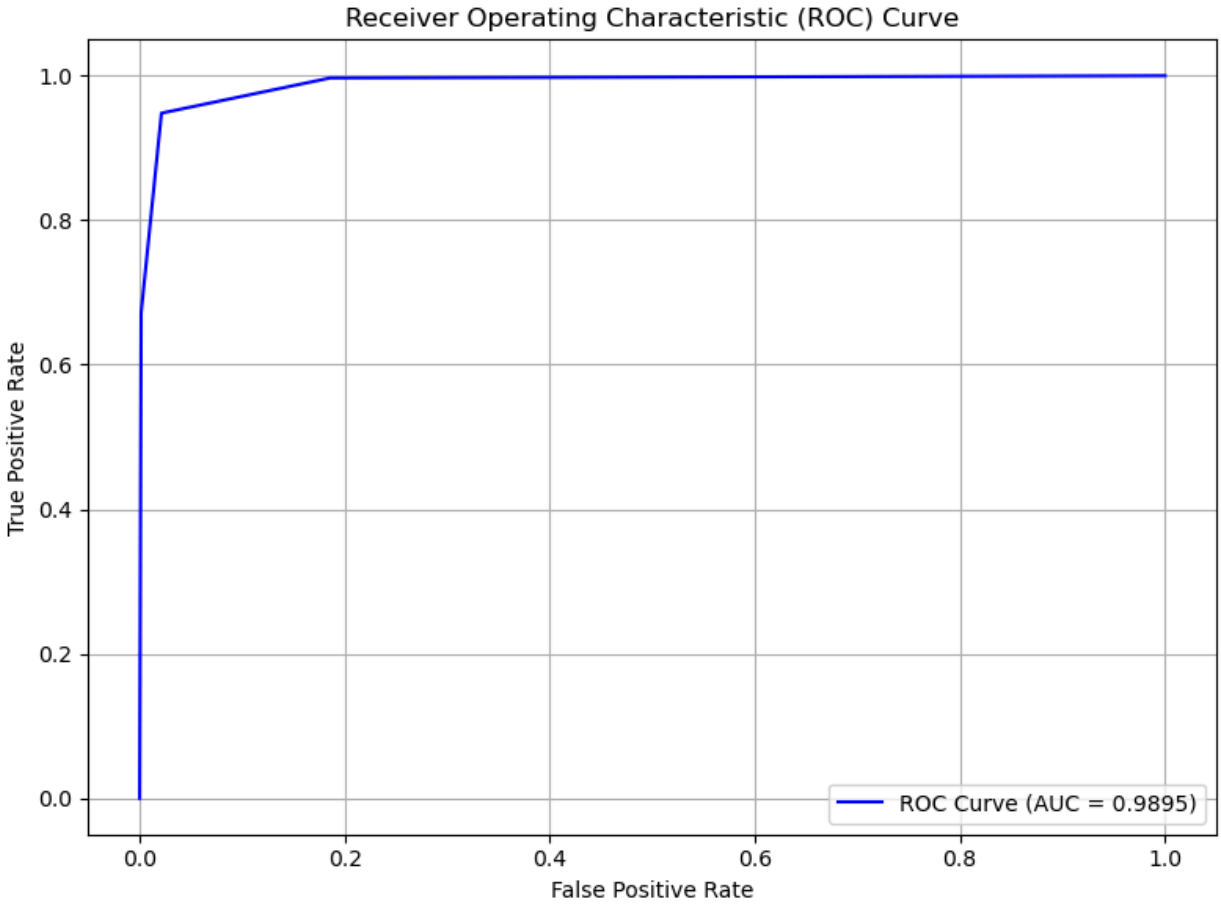


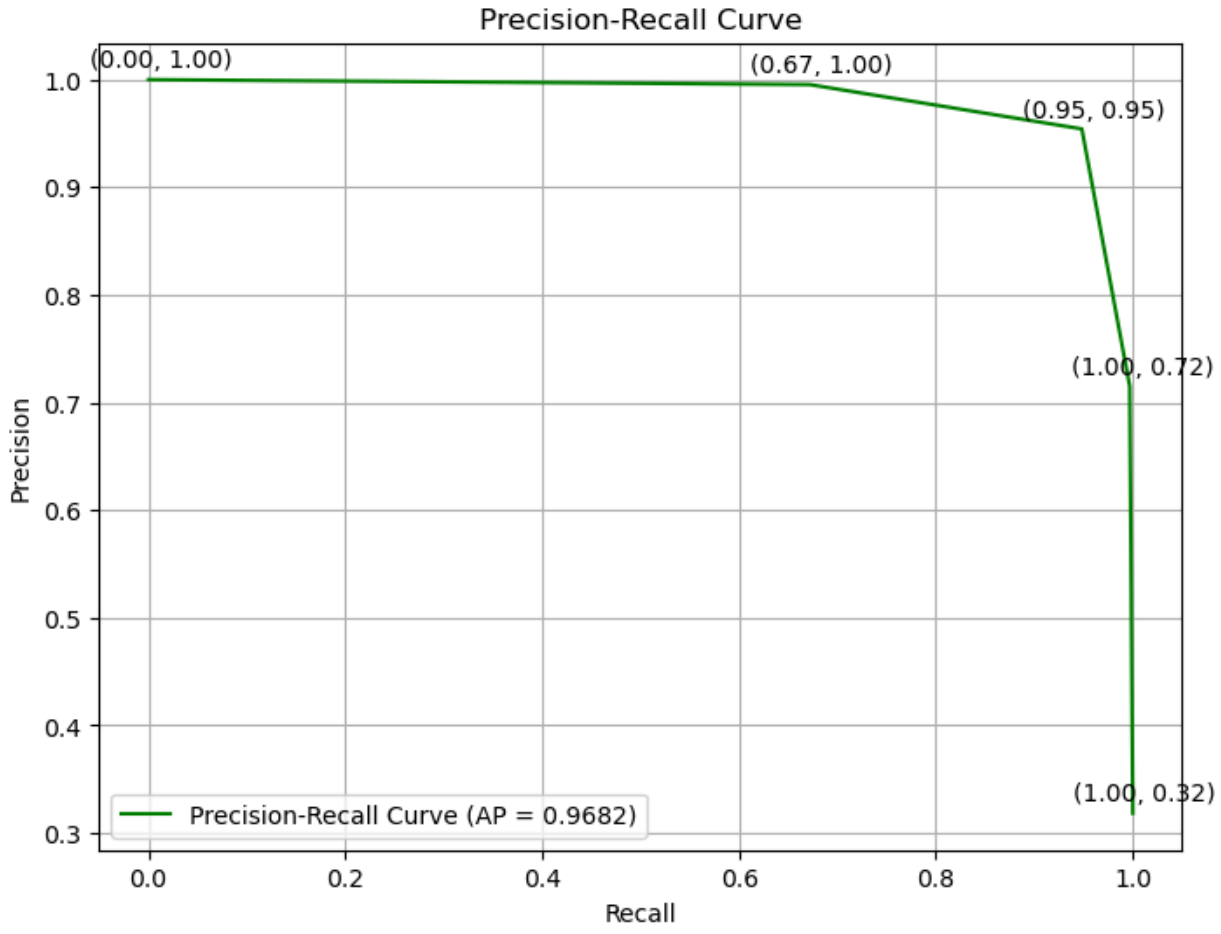
In [44]:
```
print("\nTarget :  Test Results_Inconclusive | Model: Random Forest Classifier"
print("-----------------------------------------------------------------")

print(
    classification_report(
        y_test, model.predict(x_test), target_names=["No", "Yes"], digits=6
    )
)

y_pred = model.predict(x_test)
rmse = mean_squared_error(y_test, y_pred, squared=False)
mae = mean_absolute_error(y_test, y_pred)
```

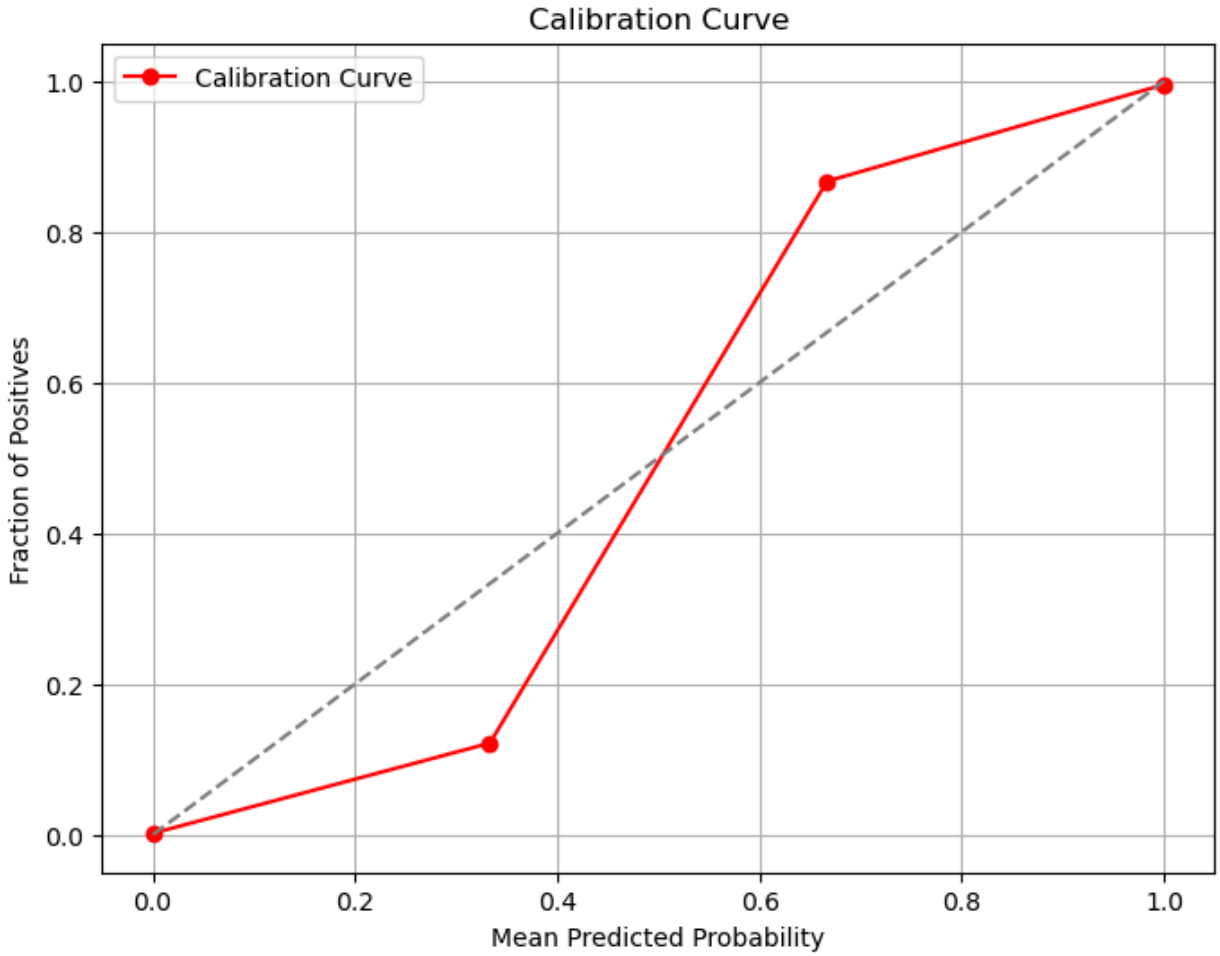
```
Target :  Test Results_Inconclusive | Model: Random Forest Classifier
-----------------------------------------------------------------
              precision    recall  f1-score   support

          No   0.975860  0.978723  0.977289      1363
         Yes   0.954186  0.948195  0.951181       637

    accuracy                       0.969000      2000
   macro avg   0.965023  0.963459  0.964235      2000
weighted avg   0.968957  0.969000  0.968974      2000
```

In [45]:
```
fpr, tpr, thresholds = roc_curve(y_test, model.predict_proba(x_test)[:, 1])
auc_score = roc_auc_score(y_test, model.predict_proba(x_test)[:, 1])

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {auc_score:.4f})', color='blue')
plt.xlabel('False Positive Rate')
```

```
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# Generate Precision-Recall Curve and calculate average precision score
precision, recall, thresholds = precision_recall_curve(y_test, model.predict_p
ap_score = average_precision_score(y_test, model.predict_proba(x_test)[:, 1])

# Calculate F-beta score
beta = 2
f_beta_score = (1 + beta**2) * (precision * recall) / ((beta**2 * precision) +

# Annotate data point values on Precision-Recall Curve
plt.figure(figsize=(8, 6))
plt.plot(recall, precision, label=f'Precision-Recall Curve (AP = {ap_score:.4f}
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.legend()
plt.grid(True)

# Annotate data point values
for i in range(len(precision)):
    plt.annotate(f'({recall[i]:.2f}, {precision[i]:.2f})', (recall[i], precisi

plt.show()


# Calculate Calibration Curve
prob_true, prob_pred = calibration_curve(y_test, model.predict_proba(x_test)[:

plt.figure(figsize=(8, 6))
plt.plot(prob_pred, prob_true, marker='o', label='Calibration Curve', color='r
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel('Mean Predicted Probability')
plt.ylabel('Fraction of Positives')
plt.title('Calibration Curve')
plt.legend()
plt.grid(True)
plt.show()
```


Receiver Operating Characteristic (ROC) Curve
True Positive Rate
False Positive Rate
ROC Curve (AUC = 0.9895)
0.0
0.2
0.4
0.6
0.8
1.0


Precision-Recall Curve
Precision
Recall
(0.00, 1.00)
(0.67, 1.00)
(0.95, 0.95)
(1.00, 0.72)
(1.00, 0.32)
Precision-Recall Curve (AP = 0.9682)
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1.0



Target : Test Results_Normal

In [46]:
```
X = df3.drop(columns=["Test Results_Normal"])


Y = df3["Test Results_Normal"]


X

Y
```

Out[46]:
```
0       0.0
1       1.0
2       1.0
3       0.0
4       1.0
       ...
9995    0.0
9996    1.0
9997    1.0
9998    1.0
9999    0.0
Name: Test Results_Normal, Length: 10000, dtype: float64
```

In [47]:
```
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.2, rando
```

```
shape_dict = {
    "Data portion": ["X", "y", "X_train", "y_train", "X_test", "y_test"],
    "Shape": [
        X.shape,
        Y.shape,
        x_train.shape,
        y_train.shape,
        x_test.shape,
        y_test.shape,
    ],
}

shape_df = pd.DataFrame(shape_dict)
shape_df
```

Out[47]:

	Data portion	Shape
0	X	(10000, 30)
1	y	(10000,)
2	X_train	(8000, 30)
3	y_train	(8000,)
4	X_test	(2000, 30)
5	y_test	(2000,)

Model: Random Forest Classifier

In [48]:
```
target = "Test Results_Normal"
model = RandomForestClassifier(n_estimators=3, random_state=42)

model.fit(x_train, y_train)

y_pred = model.predict(x_test)

accuracy = accuracy_score(y_test, y_pred)
print("\nModel: Random Forest Classifier ")
print(f"Target: {target}")
print(f"Accuracy of {target} is : ", accuracy*100,"%")
```

```
Model: Random Forest Classifier
Target: Test Results_Normal
Accuracy of Test Results_Normal is :  97.95 %
```

In [49]:
```
# Cross Validation : Model : Random Forest Classifier

cv_score = cross_val_score(model, x_train, y_train, cv=2)

print("\nTarget : Test Results_Normal | Model: Random Forest Classifier")
cv_score.mean()
```

```
Target : Test Results_Normal | Model: Random Forest Classifier
```

Out[49]:
```
0.981875
```

In [50]:
```
#Model : Random Forest Classifier
```

```
scores = cross_validate(model, x_train, y_train, cv=10, return_train_score=Tru
pd.DataFrame(scores)
```

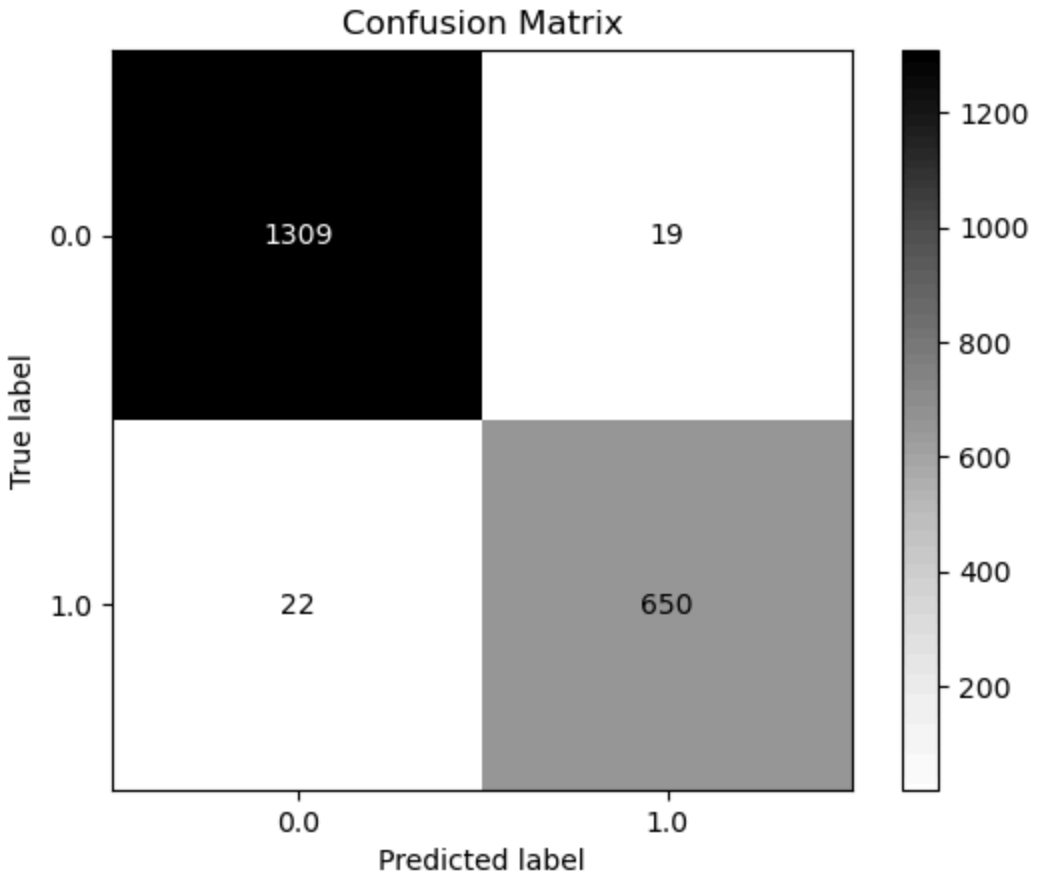
Out[50]:

	fit_time	score_time	test_score	train_score
0	0.013431	0.001356	0.98500	0.998750
1	0.014531	0.001409	0.97500	0.996389
2	0.013290	0.001376	0.99125	0.998611
3	0.013058	0.001284	0.98625	0.996944
4	0.013219	0.001315	0.99250	0.999583
5	0.013366	0.001309	0.98125	0.996528
6	0.011552	0.001213	0.99875	0.999583
7	0.012825	0.001265	0.98500	0.998056
8	0.011699	0.001204	0.99500	0.998750
9	0.012598	0.001211	0.99375	0.999167

In [51]:

```
print("\nTarget :  Test Results_Normal | Model: Random Forest Classifier")
print("-----------------------------------------------------------------")


from sklearn.metrics import ConfusionMatrixDisplay

cm = ConfusionMatrixDisplay.from_estimator(model, x_test, y_test, values_forma
plt.title("Confusion Matrix")
plt.show()
```

```
Target :  Test Results_Normal | Model: Random Forest Classifier
-----------------------------------------------------------------
```

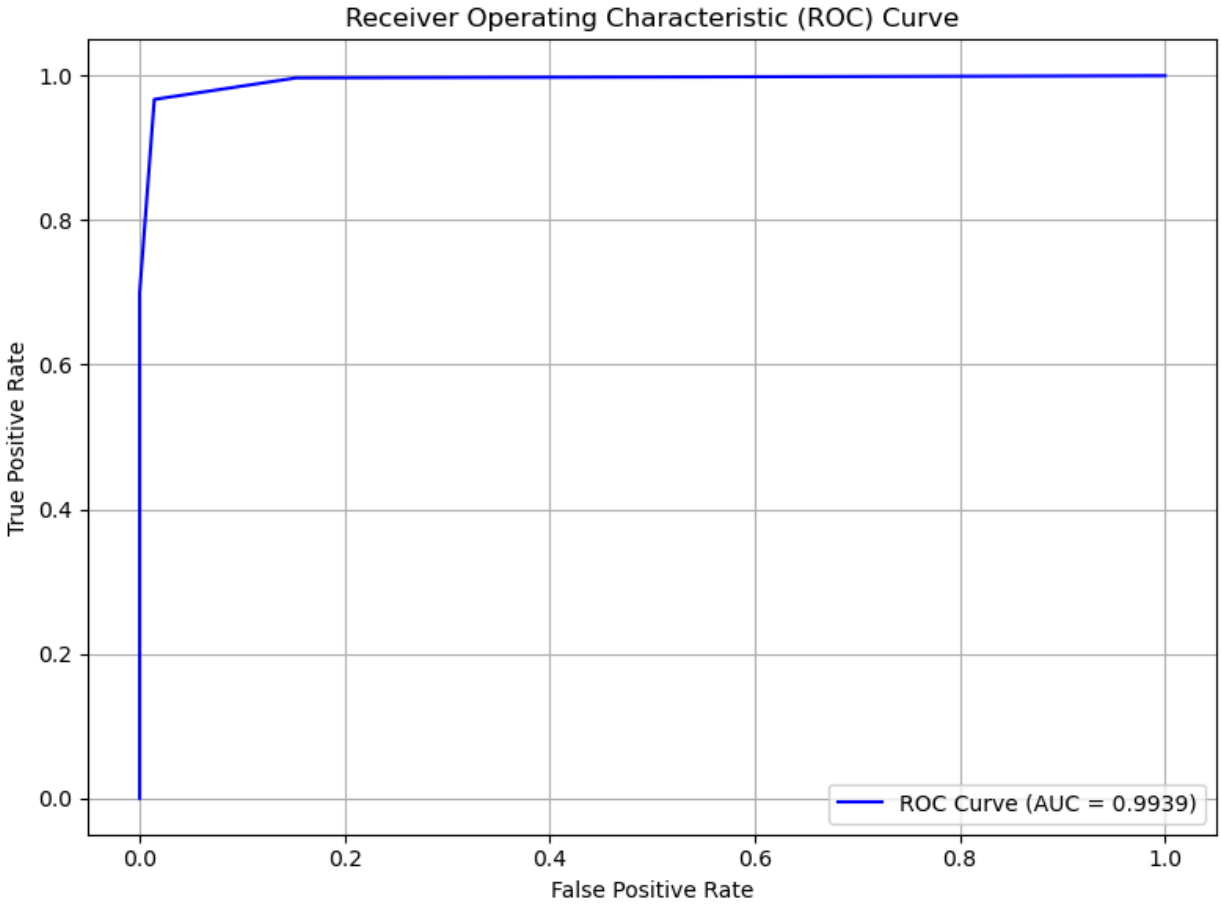


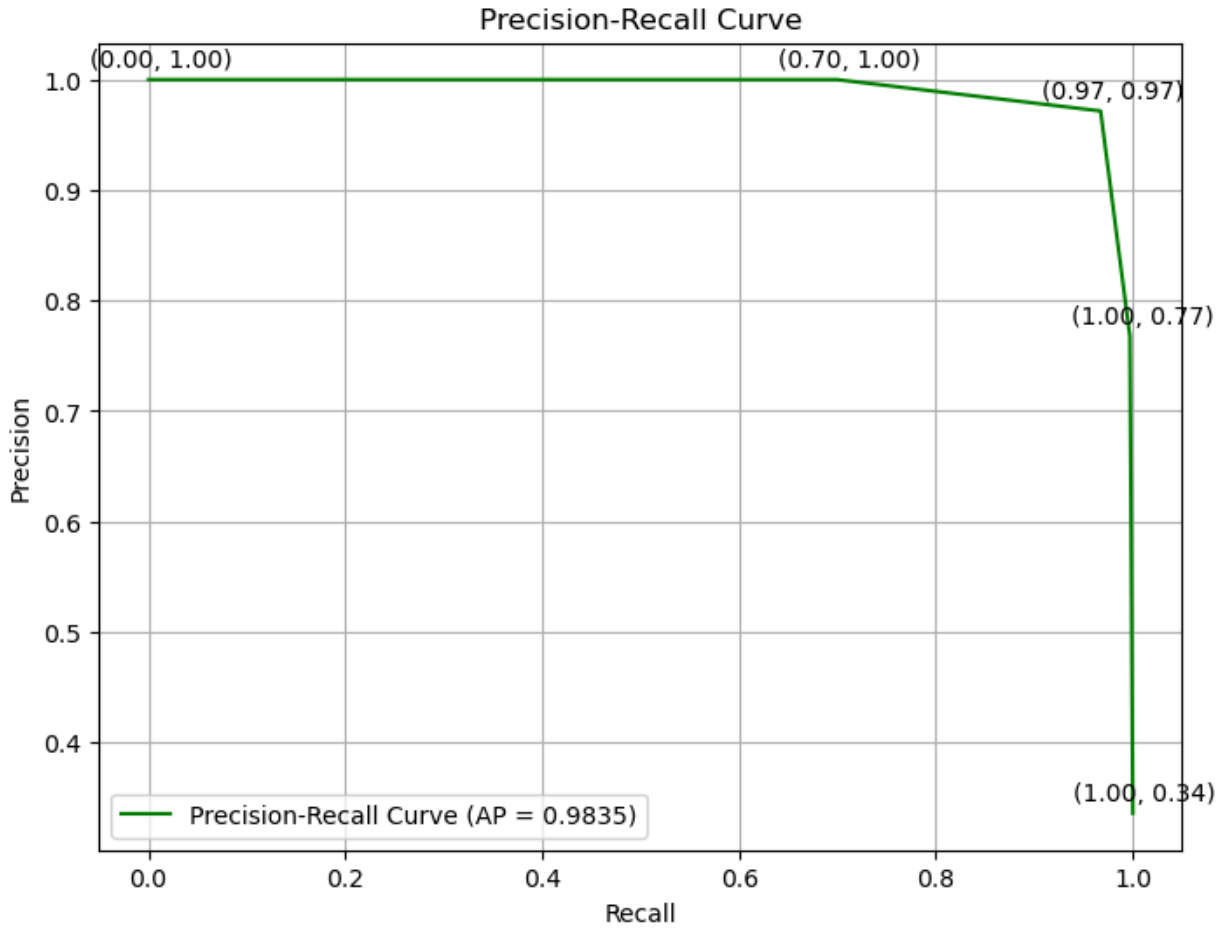
In [52]:
```
print("\nTarget :  Test Results_Normal | Model: Random Forest Classifier")
print("-----------------------------------------------------------------")

print(
    classification_report(
        y_test, model.predict(x_test), target_names=["No", "Yes"], digits=6
    )
)

y_pred = model.predict(x_test)
rmse = mean_squared_error(y_test, y_pred, squared=False)
mae = mean_absolute_error(y_test, y_pred)
```

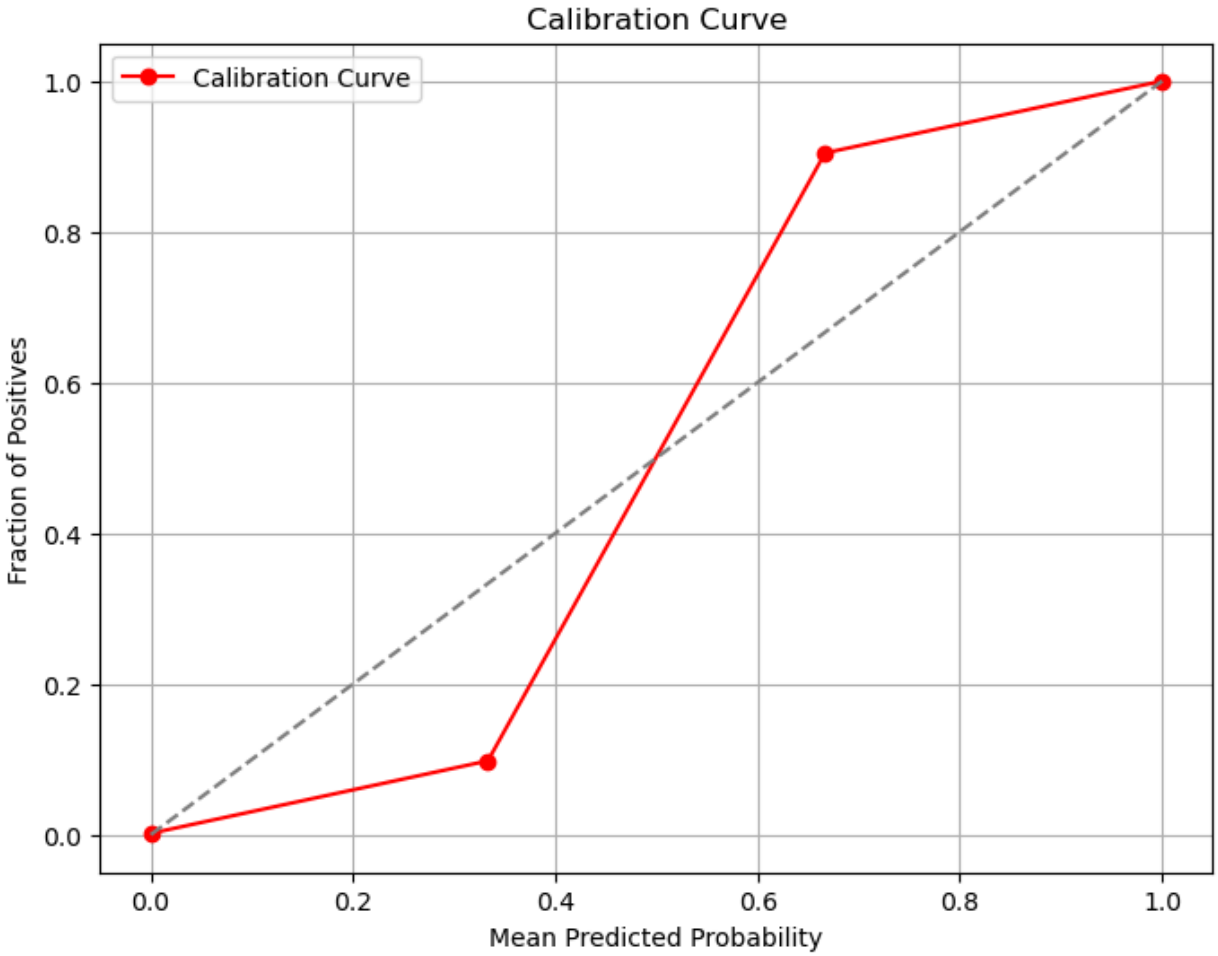
```
Target :  Test Results_Normal | Model: Random Forest Classifier
-----------------------------------------------------------------
              precision    recall  f1-score   support

          No   0.983471  0.985693  0.984581      1328
         Yes   0.971599  0.967262  0.969426       672

    accuracy                       0.979500      2000
   macro avg   0.977535  0.976477  0.977003      2000
weighted avg   0.979482  0.979500  0.979489      2000
```

In [53]:
```
fpr, tpr, thresholds = roc_curve(y_test, model.predict_proba(x_test)[:, 1])
auc_score = roc_auc_score(y_test, model.predict_proba(x_test)[:, 1])

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {auc_score:.4f})', color='blue')
plt.xlabel('False Positive Rate')
```

```
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# Generate Precision-Recall Curve and calculate average precision score
precision, recall, thresholds = precision_recall_curve(y_test, model.predict_p
ap_score = average_precision_score(y_test, model.predict_proba(x_test)[:, 1])

# Calculate F-beta score
beta = 2
f_beta_score = (1 + beta**2) * (precision * recall) / ((beta**2 * precision) +

# Annotate data point values on Precision-Recall Curve
plt.figure(figsize=(8, 6))
plt.plot(recall, precision, label=f'Precision-Recall Curve (AP = {ap_score:.4f}
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.legend()
plt.grid(True)

# Annotate data point values
for i in range(len(precision)):
    plt.annotate(f'({recall[i]:.2f}, {precision[i]:.2f})', (recall[i], precisi

plt.show()


# Calculate Calibration Curve
prob_true, prob_pred = calibration_curve(y_test, model.predict_proba(x_test)[:

plt.figure(figsize=(8, 6))
plt.plot(prob_pred, prob_true, marker='o', label='Calibration Curve', color='r
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel('Mean Predicted Probability')
plt.ylabel('Fraction of Positives')
plt.title('Calibration Curve')
plt.legend()
plt.grid(True)
plt.show()
```


Receiver Operating Characteristic (ROC) Curve
True Positive Rate
False Positive Rate
ROC Curve (AUC = 0.9939)
0.0
0.2
0.4
0.6
0.8
1.0


Precision-Recall Curve
Precision
Recall
(0.00, 1.00)
(0.70, 1.00)
(0.97, 0.97)
(1.00, 0.77)
(1.00, 0.34)
Precision-Recall Curve (AP = 0.9835)
0.4
0.5
0.6
0.7
0.8
0.9
1.0


Calibration Curve
Calibration Curve
Fraction of Positives
Mean Predicted Probability
0.0
0.2
0.4
0.6
0.8
1.0